JOURNAL OF INQUIRY-BASED LEARNING IN MATHEMATICS
No. **2**, (Mar. 2007)

Graphs and Optimization

David M. Clark

SUNY New Paltz

Contents

To the Instructor

This guide was designed for non-mathematics/science majors as a one semester college freshman level general education course in mathematics. The goal of the course is to give students a working understanding of the nature of mathematical inquiry through a study of the theory of finite graphs and their applications. As a prerequisite for this course, our students must either pass or place out of a fairly basic high school mathematics course.

The content is solid mathematics, but the topics concern very different mathematical problems and techniques than most of these students have previously encountered. As a result it creates a level playing field in which all students are starting at the beginning, and knowledge of prior mathematics is largely irrelevant.

The objects of study are **graphs**, that is, finite sets of points, called **vertices**, some pairs of which are connected by an **edge**. In this course students will see many different problems that can be modeled by graphs, and many different algorithms that can be used to solve those problems. Most of these algorithms attempt to **optimize**, that is, maximize or minimize, some quantity. These optimization problems are motivated by a variety of practical problems. In the course of this study, students learn about how to judge both the efficiency and the reliability of different algorithms.

I have taught this course with from 20 to 50 students. Larger numbers limited the opportunity for student presentations, but it remained a problem driven course focused on their own work. I presented the main topics, definitions, and theory when applicable. I asked students to present solutions to key problems on the board, sometimes letting several put up their solutions simultaneously. I asked that they each compile a neatly written portfolio which contained a complete and final solution to each of the 71 numbered problems listed in the text. Some days were spent entirely with them working at their desks on problems and me walking around or sitting in front and answering individual questions that they brought to me. I required that they turn the portfolio at the end of each of the 5 chapters, but encouraged them to bring it to me more often if they liked. I marked errors and allowed them to revise as many times as it took to get every problem done right.

Based on our experience teaching from this text over a number of years, we have found that the 5 chapters and 71 problems given here are a comfortable but full quantity of mathematics to fill a one semester course.

A sequence of stated theorems and propositions forms the theoretical backbone of this course. However this is not intended as a theorem proving course in the sense that I do not expect students to prove the theorems themselves. Instead I present a “proof” through a sort of Socratic method; an interactive conversation with the students that results in a rough argument as to why it is true. Because the topic is finite graph algorithms, these

"proofs" are usually based on a reasonably complex example that I draw on the board. I expect the students to come away from this experience with a sense that theorems are mathematical statements that can be justified by a rational argument, though one that they would be unlikely to be able to sit down and reproduce. For the students in these classes, justification of the theorems comes to them by working the problems in which they see for themselves that the statements are true in concrete examples. The same text might provide a basis for a more advanced student to work out careful proofs of these results, but I have not used it in this way.

I give only two exams, a midterm and a final, where students have ample time for each. Their final grades are calculated as 25% midterm exam, 25% final exam and 50% portfolio. No partial credit is given for the portfolio; it is either 100% correct or earns no credit at all. The result is that there are very few students who do not keep up the portfolio. Those students not only have poor portfolio grades but are also not able to do the exams. They usually recognize the situation and drop the course before the midterm deadline. Most of the students do keep up with the course as we go, complete the portfolio, and are well prepared for the exams.

Altogether I found it to be a successful course that students seemed to enjoy, and that left them with a very different view of mathematics than they brought with them from high school.

David Clark
clarkd@newpaltz.edu
March 2007

Acknowledgements

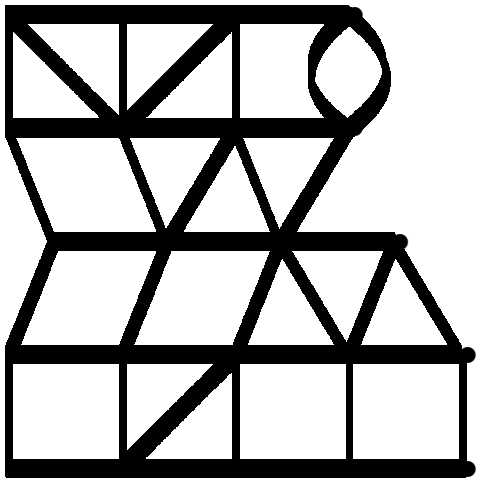
This guide was written under the auspices of SUNY New Paltz. The author gratefully acknowledges Allison Vandenbrul and Anthony Foppiano for the fine graphic work, Harry Lucas of the Educational Advancement Foundation for his financial support that allowed them to do it, and Jane Conger and Barbara Heiles who provided many useful suggestions after teaching repeated sections from earlier drafts. Finally, many helpful comments from the referee resulted in a much refined product.

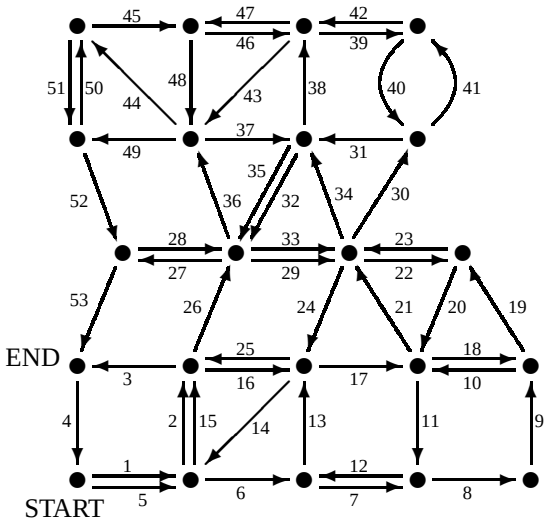
DMC

Chapter 1

Euler Paths and Circuits

We will begin this chapter with a practical problem. Suppose that you are offered a job delivering mail in the neighborhood shown in the street diagram on the left. Each segment represents one block along which you will need to deliver mail, and requires 10 minutes to walk.





The Postmaster suggests you take 70 minutes for lunch and breaks plus 10 minutes for each of the 41 blocks (count them!) shown, a total of

$$70 + 41 \times 10 = 70 + 410 = 480 \text{ minutes}$$

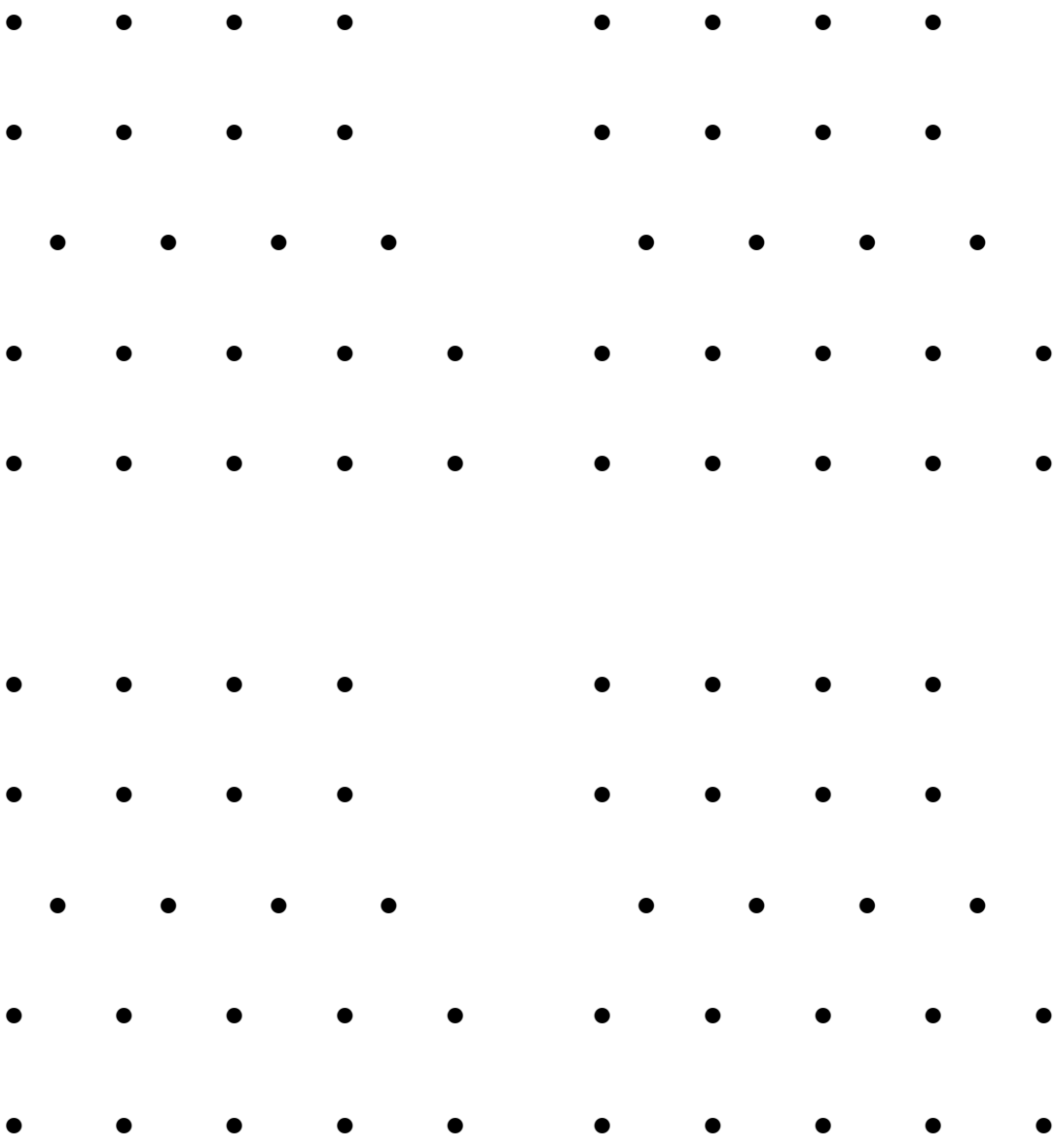
or 8 hours each day. Does this sound like a good offer?

The first thing you need to decide is, what path should you follow? For example, one possible path is shown on the right above. This path requires walking 53 blocks, which will take you

$$70 + 53 \times 10 = 70 + 530 = 600 \text{ minutes},$$

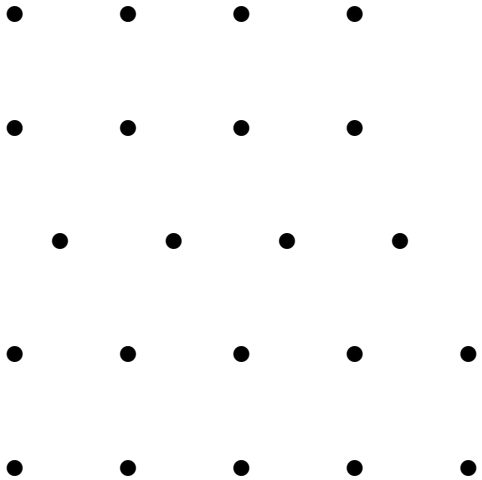
or 10 hours a day! This is definitely not a good proposition. Can you do better? Your goal is to find the shortest path, in terms of the number of blocks you walk, that traverses each block at least once. You can start and end where ever you like.

Experiment with the diagrams below to find the shortest path you can. How long, in hours and minutes, will it take you to finish if you include 70 minutes for lunch and breaks?



Problem 1. *Write down your best solution to the above Mail Carrier Problem, labeling the START and END and numbering the blocks as you go.*

1. *How many blocks must you walk?*
2. *How long* (*in hours and minutes*) *will it take you?*



As we can see, it is difficult to tell how long the absolute shortest path will be through this neighborhood. It is at least 41 blocks, and it is no longer than your answer to 1. above. In order to better understand these kind of problems, examine the mini-neighborhoods illustrated on the next page by a schematic diagram showing streets as lines and intersections as solid circles. Copy each one on another paper and experiment with different paths. Then, in each case, write down

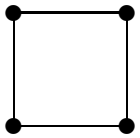
A. the total number of blocks in the neighborhood,

B. the time required, at 10 minutes per block, if each block were travelled exactly once,

C. the smallest number of blocks that you can find to deliver mail along each block (show your path) and

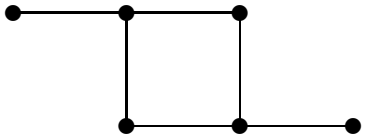
D. the time required for (C.), at 10 minutes per block.

Problem 2.



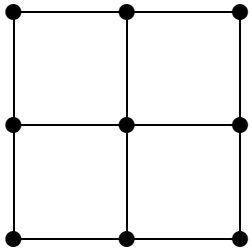
A =
B =
C =
D =

Problem 3.



A = *B* = *C* = *D* =

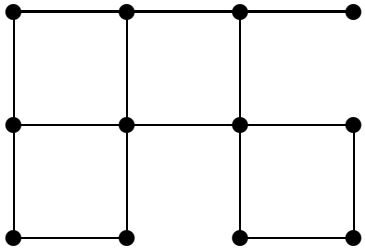
Problem 4.



A = *B* =

C = *D* =

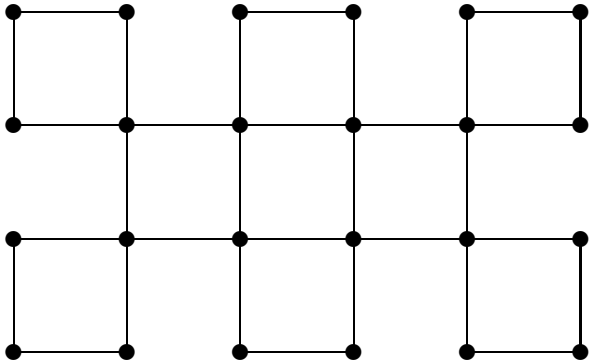
Problem 5.



A = *B* =

C = *D* =

Problem 6.



A =

B =

C =

D =

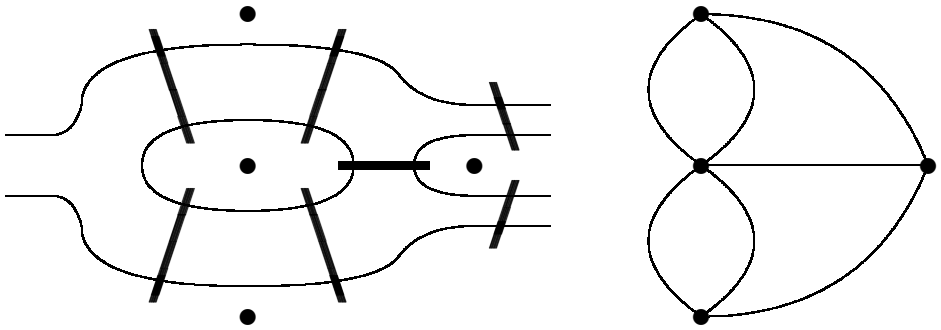


Figure 1.1: Königsberg Bridges

Mail Carrier Problems, and the mathematics we will learn to solve them, have an interesting history which goes back to eighteenth century east Prussia. There the historic city of Königsberg (now Kaliningrad) is broken into four pieces by the forking Pregel River. Over the river seven bridges connect the parts of the city as shown in Figure 1.1. On leisurely Sunday strolls, the folks of Königsberg found an amusing challenge: could they find a path which would take them over each bridge exactly once? If you aren't planning to visit Kaliningrad any time soon, you might try finding such a path with a pencil by sketching the bridge configuration on another paper. Can you find one?

You might notice that this looks vaguely like a Mail Carrier Problem. In fact, both kinds of problems have a common formulation. Suppose we identify 4 particular landmarks, one on each landmass, and 7 particular pathways, one over each bridge, connecting them. The resulting diagram is shown on the right. If we imagine that this diagram represents a mail route, then the Königsberg Bridge Problem, stated above, is equivalent to the problem of finding a path taking us down each street *exactly once*. Do you think this is possible? If not, what is the shortest path that passes over each street?

The original Mail Carrier Problem asks that we find the shortest possible path through a neighborhood which passes over each of 41 residential blocks. This path must be at least 41 blocks long, but our experiments suggest that the shortest path will have to be somewhat longer. Similarly, the Königsberg Bridge Problem asks that we find a path that passes over each of 7 bridges exactly once. Again, it appears that any path passing over each bridge at least once will go over some bridge twice.

Mathematics models problems like these using something called a "graph". The diagrams we used to represent Mail Carrier Problems and the diagram on the right of Figure 1.1 are examples of graphs. Graphs were invented by Leonard Euler, an eighteenth century resident of Königsberg, in order to solve the Königsberg Bridge Problem. Below is a list giving exact definitions of the important notions that Euler discovered.

- A **graph** consists of a set of points, called its **vertices**, and a set of line segments connecting them, called its **edges**.

A graph can be drawn in different ways. In Figure 1.2 we see two drawings of the same

graph. While the one on the left may be more artistic, the one on the right makes it easier to recognize connections between the six vertices and the six edges.

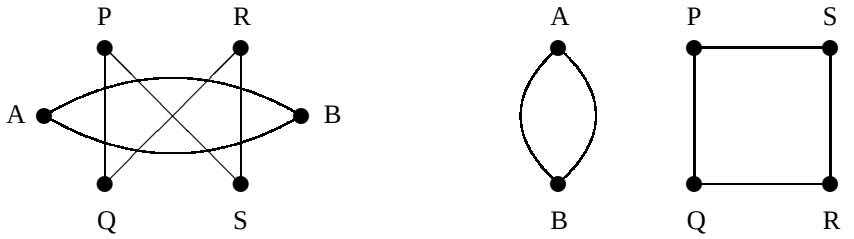


Figure 1.2: Two ways to draw the same graph.

- A **loop** is an edge connecting a vertex to itself. Two or more edges between the same two vertices are called **multiple edges**.
- Two vertices are **adjacent** if an edge connects them.
- A **path** is a sequence of adjacent vertices with a connecting edge between each pair of vertices.
- A **circuit** is a path which starts and ends at the same vertex without repeating an edge.
- An **Euler path** is a path that passes over every edge of the graph exactly once.
- An **Euler circuit** is a circuit that passes over every edge of the entire graph.

To solve Mail Carrier Problems and the Königsberg Bridge Problem, we will first try to decide when it is possible to find a path going over each block or each bridge *exactly once*. These problems can both be rephrased as the problem of determining whether or not the associated graph has an Euler path. Several additional notions will be useful to do this.

- The **degree** of a vertex in a graph is the number of edges coming into it. (A loop counts as two edges.) A vertex is **even** if its degree is an even number, and it is **odd** if its degree is an odd number.
- A graph is **connected** if every pair of vertices is connected by some path. A **component** of a graph is a maximal connected piece of the graph.
- An edge of a connected graph is called a **cut edge** if removal of that edge causes the graph to become disconnected.

Exactly six of the follow seven problems are possible to do. The seventh one is not!

Problem 7. *Draw a graph which has 6 vertices, 4 loops and 2 multiple edges.*

Problem 8. *Draw a graph with 5 vertices, each of degree 4, which has no loops or multiple edges.*

Problem 9. *Draw a graph with 3 vertices of degree 2 and 2 vertices of degree 3.*

Problem 10. *Draw a graph with 2 vertices of degree 2 and 3 vertices of degree 3.*

Problem 11. *Draw a graph with 1 vertex of degree 1, 2 vertices of degree 2, 3 vertices of degree 3 and 4 vertices of degree 4.*

Problem 12. *Draw a graph with 8 vertices and 6 edges which contains no circuit at all.*

Problem 13. *Draw a connected graph with exactly 3 edges that does not have an Euler path.*

Using the definitions of these terms, the facts below follow logically, one from the other. They tell us exactly which graphs have an Euler path or an Euler circuit and how, if there is one, to find it. (Remember that an Euler circuit is a special kind of Euler path.)

Odd Vertex Theorem. *Suppose A is an odd vertex of a graph.*

1. *An Euler path that starts at A cannot end at A.*
2. *An Euler path that does not start at A must end at A.*
3. *Every Euler path either starts or ends at A.*

Even Vertex Theorem. *Suppose B is an even vertex of a graph. Then every Euler path that starts at B must also end at B* (*and is therefore an Euler circuit*)*.*

From these two observations we can quickly establish the following necessary conditions for a graph to have an Euler path or an Euler circuit.

First Euler Path Theorem. *If a graph has an Euler path, then*

1. *it must be connected and*
2. *it must have either 0 or 2 odd vertices.*

First Euler Circuit Theorem. *If a graph has an Euler circuit, then*

1. *it must be connected and*
2. *it must have no odd vertices.*

The two theorems above tell us which graphs do not have an Euler path or circuit. We would like to demonstrate that all remaining graphs—connected graphs with either 0 or 2 odd vertices—do have Euler paths. To do so requires some auxiliary ideas. A **mouse** is a section of a connected graph (**the body**) which is connected to the rest of the graph by a single cut edge (**the tail**).

Proposition 1. *Every graph has an even number of odd vertices.*

Proof: The sum of the degrees of the vertices of a graph is twice the number of edges, an even number. Since the sum of even numbers and an odd number of odd numbers is odd, the number of odd vertices must be even. #

Proposition 2. *The body of every mouse contains an odd vertex.*

Proof: Think of the mouse as a graph by itself. Then the tip of its tail is one odd vertex so, by Proposition 1, there must be at least one more odd vertex in its body. #

Fleury's Theorem. *If a graph is connected and has either 0 or 2 odd vertices, then an Euler path can be found by the following algorithm.*

1. *Start at one of the odd vertices if there are any; otherwise start where ever you choose.*
2. *From each vertex in the path, traverse any untraversed edge that is not a cut edge. Only traverse a cut edge if there is no other choice.*
3. *Stop where you reach a vertex which is not adjacent to any untraversed edge, that is, when you can't go any further without repeating an edge.*

At that point you will have completed an Euler path.

Proof: Assume there are two odd vertices, A and B. (A very slight variation of this argument works if there are no odd vertices.) Start Fleury's Algorithm at A and observe each of the following.

1. At each step in the path, either we are at B and there are no odd vertices in the remaining graph or we are at a vertex X other than B and B and X are the only two odd vertices in the remaining graph.
2. If we go from a vertex X to a vertex Y, erasing the connecting edge, then Y and the remaining edges still form a connected graph.
3. When we reach a vertex with no remaining edges (degree 0), that vertex must be B and all of the edges must have been erased. #

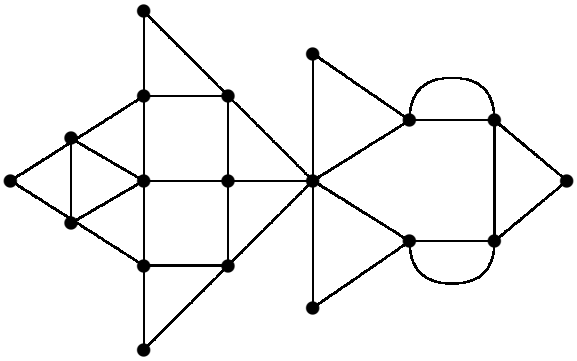
Fleury's Algorithm will be demonstrated in class using the graph in Figure 1.3. Applying Fleury's Algorithm and Fleury's Theorem, we see that the following two conclusions following immediately.

Second Euler Path Theorem.
If a graph is connected and has exactly 2 odd vertices, then it has an Euler path.

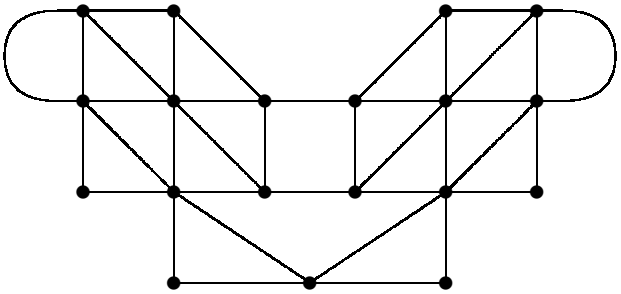
Second Euler Circuit Theorem.
If a graph is connected and has no odd vertices, then it has an Euler circuit (which is also an Euler path).

You can now assemble all of these ideas in order to see whether or not each of the following three graphs has an Euler path or an Euler circuit.

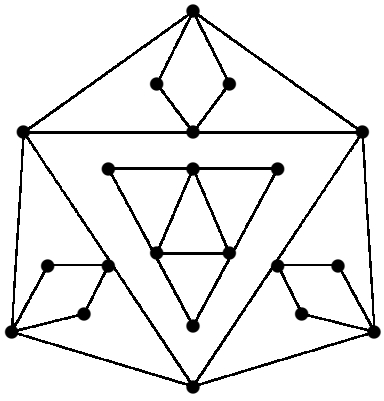
- If it has an Euler path or Euler circuit, trace it on the graph by marking the START and END and numbering the edges.
- If it does not, then write a *complete sentence* explaining how you know it does not.



Problem 14.

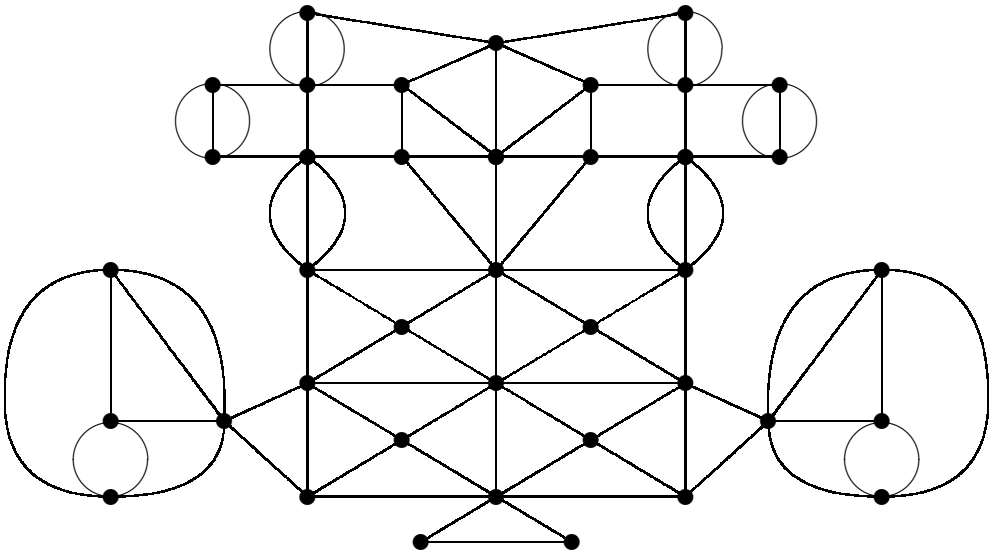


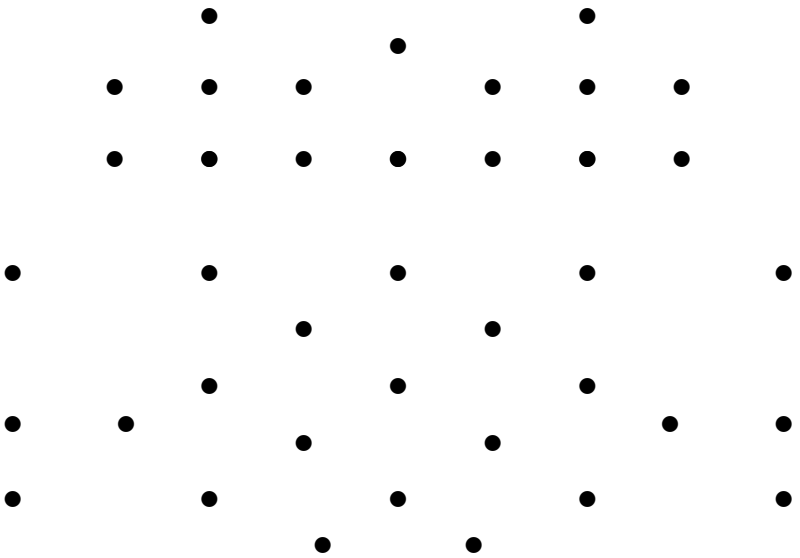
Problem 15.



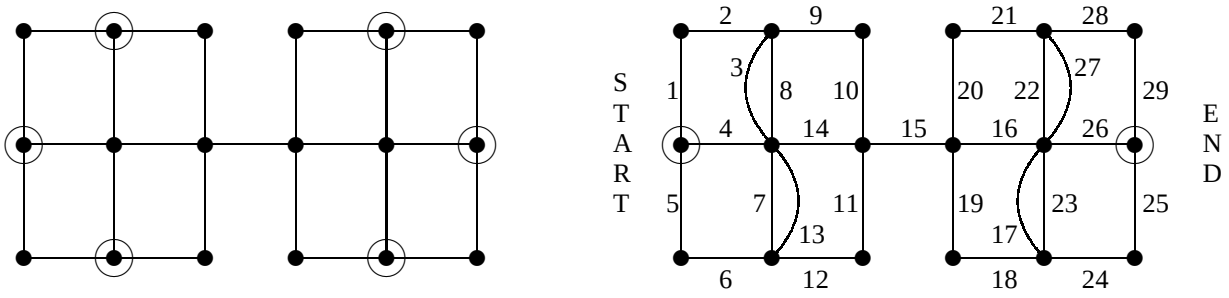
Problem 16.

Problem 17. *Use Fleury's Algorithm to find an Euler path for the graph below.* (*Draw the graph again in pencil at the bottom. Then number the edges of the Euler Path on top, erasing each edge on the bottom as you traverse it. Label the "START" and "END" of your path on the upper graph.*)





Now that we know exactly which graphs have a path or circuit passing over each edge exactly once, we can try to apply these ideas to our original mail carrier problems. Consider the problem of delivering mail in the small neighborhood illustrated on the left below. It has 25 blocks along which mail must be delivered.



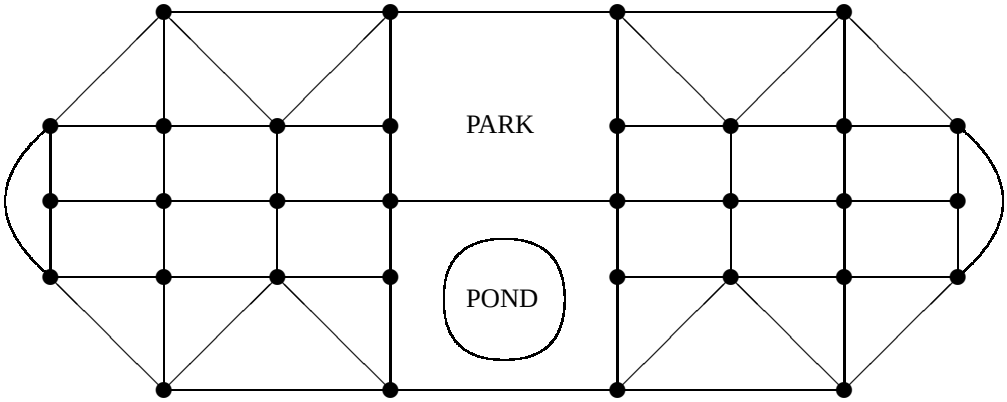
We can quickly check that there are exactly six odd vertices, and conclude that there is no Euler Path, that is, there is no way we can deliver all the mail by walking down each block exactly once. But the mail still has to be delivered! What is the smallest number of blocks that we can walk and still deliver all the mail?

We can answer this question by the process of **Eulerization** of the graph. We insert in the graph the smallest possible number of multiple edges that will reduce the number of odd vertices to 2. This is done in the graph on the right. The smallest number of multiple edges that will reduce the number of odd vertices to 2 is, in this example, 4. The edges in the new graph not only represent streets; they represent particular paths that we will walk. A multiple edge is a block that we will walk down twice. Since this graph is connected and has only 2 odd vertices, it has an Euler Path. The Euler Path represents a path that we can actually walk, multiple edges being the blocks we walk twice. Since we added 4 multiple edges, we walk a total of $25+4=29$ blocks. This is the shortest path that will cover each block in the neighborhood.

WARNING! When you use the method of Eulerization, be sure that you only add multiple edges which duplicate existing edges. You can't add new edges between vertices that were not previously connected. (This is because residents generally complain if the Post Office starts bulldozing new streets through their back yards in order to efficiently deliver the mail!)

Now apply this method to the original Mail Carrier Problem. There are 41 blocks in the neighborhood. What is the minimum number of blocks that must be walked in order to deliver all the mail?

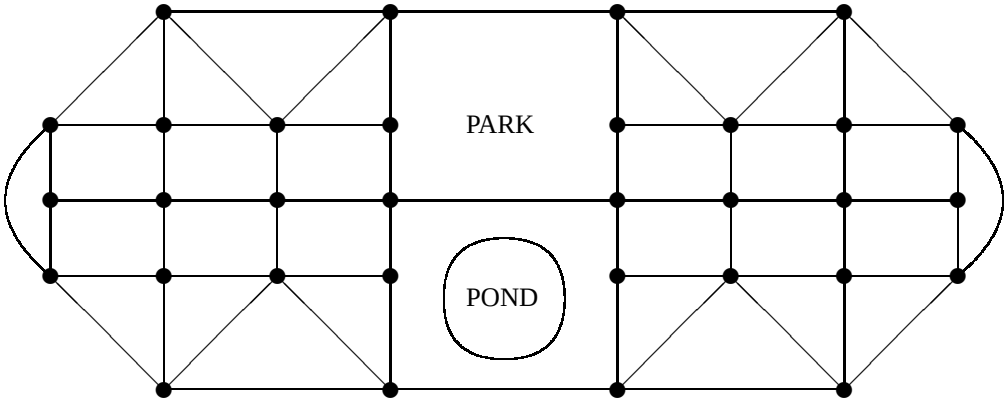
In the following problems, we will put together everything that we have learned. Imagine that a friend of yours recently took a job as Postal Carrier delivering mail to the community shown in the street map below. The map shows 63 blocks (not all the same size) that must be traversed in order to deliver all of the mail. Never having studied graph theory, he starts each day at a randomly selected intersection and follows one street after another, only retracing streets when it is necessary, until mail has been delivered to every street.



Problem 18. *Trace out a possible path that he might follow. Mark the START and END, and number the blocks in the order that he walks them. Of course, the blocks that he retraces will get two numbers. Mark with a colored pen or marker those blocks that he traverses twice. What is the total number of blocks that he walks?*

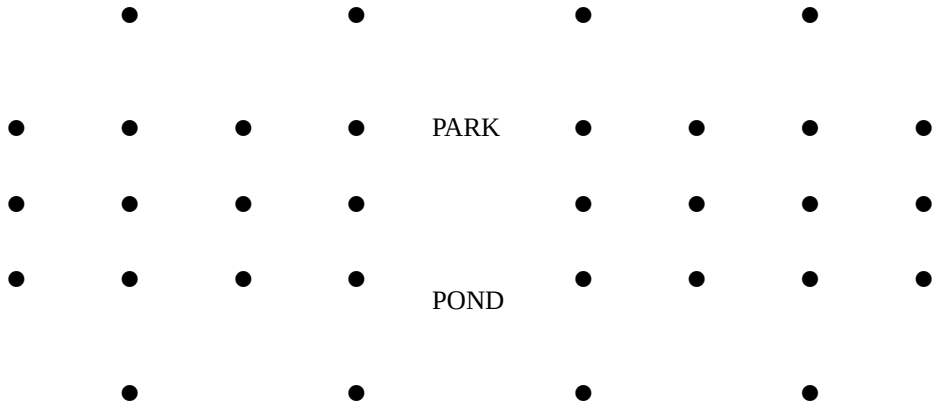
Number blocks he walks:

Problem 19. *One day he comes to you looking tired and bedraggled, and complains that he's working long days and just can't get the job done in the time he is expected to do it. See if you can help him find an optimal path. Circle the odd vertices in the graph below. Then find an optimal Eulerization of this graph by adding as few duplicate edges as are absolutely necessary to produce a graph with only two odd vertices.*

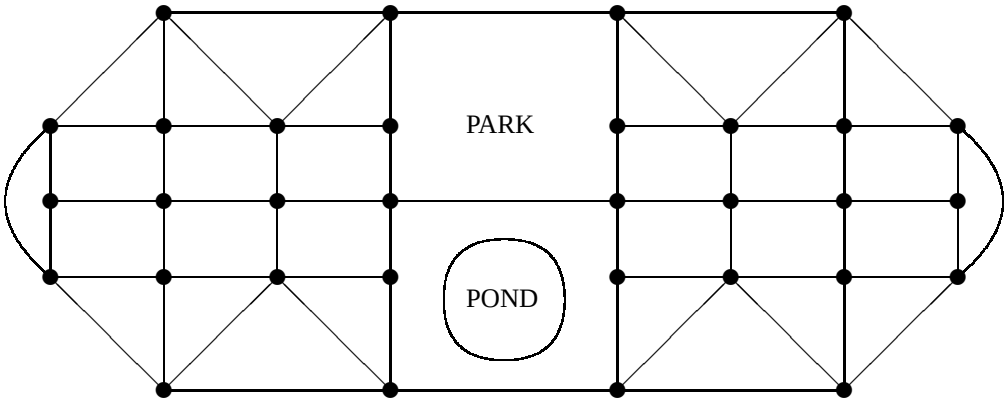


Problem 20. *Using the dots below, copy the Eulerized graph above lightly with a pencil. Now use Fleury's algorithm to find an Euler path for this graph. Start at an odd vertex; number edges as you traverse them in the upper graph and erase them in the lower graph. Meander where ever you like, making sure only that you don't cross a bridge until you have*

to. Use the lower graph to identify the bridges. If you do it right, you will end at the second odd vertex with an Euler Path. Be sure to mark the START and the END.



Problem 21. *Finally, use your to the previous problem to create a new path for your friend in the graph below. As before, mark the START and END, and number the blocks in the order that he is to walk them. Then mark with a colored pen or marker the blocks that he will traverse twice. What is the total number of blocks that he must walk now?*



Number blocks he walks:

Put this assignment in your portfolio, and be sure to give a copy to your friend the Postal Carrier.

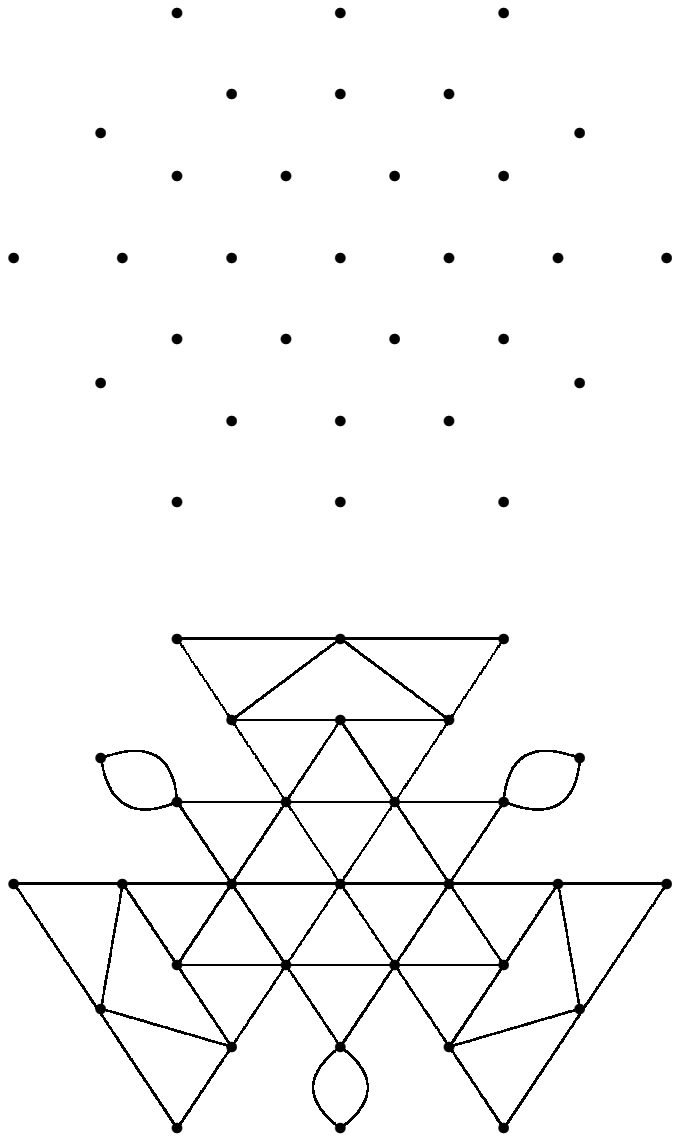


Figure 1.3: Fleury's Algorithm

Chapter 2

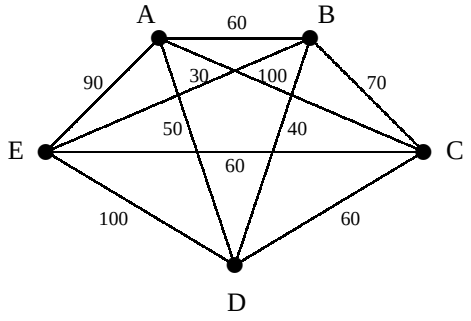
Traveling Salesman Problems

Imagine that you return to the Postmaster and explain to him that the mail cannot be delivered in 8 hours, as he suggests. You have studied graph theory and know now, for sure, that the shortest route will traverse 47 blocks and will therefore take you $70 + 47 \times 10 = 540$ minutes, or 9 hours per day. To your dismay, you discover that the Postmaster doesn't understand the first word about graph theory and continues to insist that you should be able to do the job in 8 hours. So you pack up and look for another job.

A local health food supplier is looking for someone to make deliveries to stores in five towns in the area. He shows you a schematic diagram of the five towns, labeled A, B, C, D and E, with the distances between them indicated in miles, and says that you will be paid for each round trip. You can start tomorrow. You'll leave A, where you live, go to B, then C, then D, then E and back to A. That's just

$$60 + 70 + 60 + 100 + 90 = 380 \text{ miles},$$

which you can certainly do in a day, so you can make one round trip each day.



Why this looks great, you say; you can travel at your own pace, and you *surely* won't have to worry about graph theory any more! But 380 miles a day, on country roads, with 5 stops. Hmm.... Of course, you *could* go from A to B to D to E to C and back to A. That's just

$$60 + 40 + 100 + 60 + 100 = 360 \text{ miles},$$

which is 20 miles shorter. Now you start to wonder, what is that the shortest route? How many routes are there?

These questions lead you into a new area of graph theory. A **weighted graph** is a graph in which each edge has been assigned a positive number called its **weight**. The graph above is an example of a weighted graph. The vertices represent the towns and the weights represent the distances between towns. We are interested in the circuits in this graph which go through *each vertex exactly once*. Such a circuit is called a **Hamilton circuit**. A **minimum weight Hamilton circuit** in a weighted graph is a Hamilton circuit that has the smallest possible weight of all Hamilton circuits.

The **Traveling Salesman Problem** (**TSP**) is the problem of finding a minimum weight Hamilton circuit in a weighted graph. This graph problem can be used to model a variety of different real world problems, for example:

package deliveries, where vertices represent locations and weights represent travel distances;

errands, where vertices represent locations and weights represent travel times;

work orders, where vertices represent repair jobs and weights represent times required to re-tool for the next job;

jobs on a machine, where vertices represent tasks and weights represent times to reconfigure the machine for the next task;

circuit boards, where vertices represent holes to be drilled in the board and weights represent times to rotate the board into the new position.

Usually the vertices of a weighted graph represent locations or configurations and the weights represent distances between locations or times required to change configurations. In each of these examples a solution to the TSP for the weighted graph gives us the most efficient way to do the entire series of tasks.

There is a straightforward **brute force algorithm** to solve the TSP in small examples like the one above. We simply make one list of all possible Hamilton circuits, and give the total distance required to follow each one. (Notice that the circuit ABCDEA is the same as the circuit CDEABC, so we list each one starting at A.)

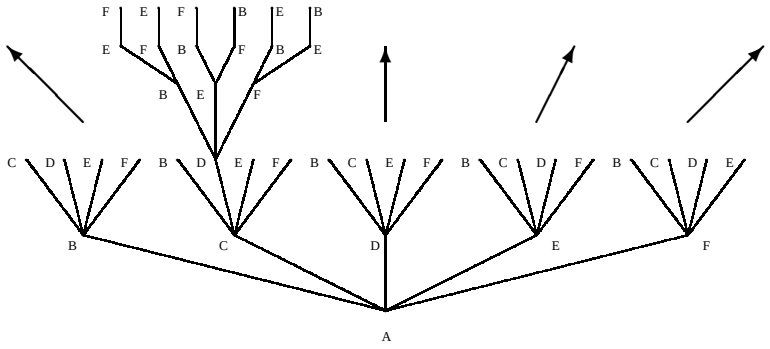
	ABCDEA, AEDCBA: 380 mi.	ACBDEA, AEDBCA: 400 mi.
	ABCEDA, ADECBA: 340 mi.	ACBEDA, ADEBCA: 350 mi.
*	ABECDA, ADCEBA: 260 mi.	AEBCDA, ADCBEA: 300 mi.
	ABDCEA, AECDBA: 310 mi.	ADBCEA, AECBDA: 310 mi.
	ABDECA, ACEDBA: 360 mi.	ADBECA, ACEBDA: 280 mi.
	ABEDCA, ACDEBA: 350 mi.	AEBDCA, ACDBEA: 320 mi.

As we can see from this list, the route you choose makes a big difference in the distance that you must travel. The longest Hamilton circuit is ACBDEA, which is 400 miles, while the shortest Hamilton circuit is ABECDA, a mere 260 miles. Wow! By solving the TSP for this weighted graph, you've made a difficult looking job into a very easy one. Okay, you say, I'll take the job!

A special kind of graph is used to model most instances of the TSP. A **complete graph** is a graph in which every pair of distinct vertices is connected by an edge. The above graph is the complete graph on 5 vertices.

Problem 22. *Draw the complete graph on 6 vertices, A, B, C, D, E and F.*

To apply the brute force algorithm, you need to find all of the different Hamilton circuits. How many are there? A useful way to count them is to draw a tree diagram. Look at your complete graph on 6 vertices. Starting with A, there are 5 places we can go: B, C, D, E or F.



From any one of these, say C, there are 4 places we can go, B, D, E or F, since you have already been to A. From D we can continue to any of 3 places: B, E or F. From F we have 2 choices: B or E, and from each of these there is only 1 choice. The total number of Hamilton circuits in this complete graph with 6 vertices is therefore $5 \times 4 \times 3 \times 2 \times 1 = 5! = 120$. (Recall that $N!$, read "N factorial", is the product of the first N positive numbers: $N! = N \times (N-1) \times \cdots \times 3 \times 2 \times 1$.) We can state this result in general.

Proposition 3. *A complete graph with N vertices has exactly* $(N-1)!$ *Hamilton circuits, half of which are duplicates obtained by reversing those in the other half.*

We will study three different algorithms for solving the TSP, each of which has advantages and disadvantages. The above solution is an example of our first algorithm.

Brute Force TSP Algorithm.

1. *Draw a tree of all* $(N-1)!$ *Hamilton circuits.*
2. *Compute the total weight of each circuit in the list.*
3. *Pick out a minimum weight circuit.*

It is easy to see that this algorithm always gives the optimal solution, since it requires that we check every possible solution. The problem with the Brute Force Algorithm is that it is terribly inefficient since the time it takes to execute grows much too fast as we add new vertices. For example, suppose we have a weighted graph with 25 vertices. To decide if this graph has an Euler path, and then to use Fleury's Algorithm to find that Euler Path, might take as much as 20 minutes. But checking 24! different Hamilton circuits is a different story. A computer that can check 500,000 circuits each second would take approximately 15 billion years (more than the age of the universe) to complete the Brute Force Algorithm!

This leads us to look for better algorithms to solve the TSP. We will look at two other algorithms for solving the Traveling Salesman Problem which are both quick and efficient. They are examples of **greedy algorithms** because they ask that we do at each step whatever is most advantageous to us at that moment, with no attempt to plan ahead.

Nearest Neighbor Greedy Algorithm. *Apply the following three steps to make a circuit starting from each vertex:*

1. *Start at any vertex.*
2. *At each step, go to any one of the closest of the remaining vertices.*
3. *Once every vertex has been visited, return home.*

Now choose the shortest of the circuits you obtained.

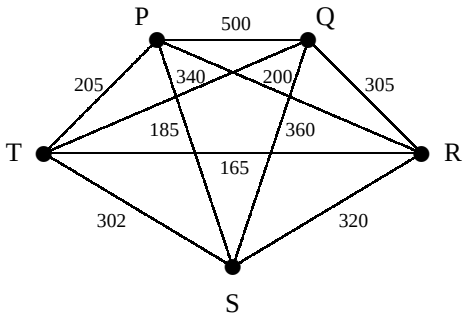
Cheapest Link Greedy Algorithm. *Add edges, one at a time, to form a Hamilton circuit as follows.*

1. *Choose any edge with the least weight.*
2. *As long as there remain unused edges whose addition*

   (a) *would not create a degree three vertex and*

   (b) *would not close a circuit,*

   add one of minimum weight.
3. *When step* 2. *can not be continued, add the edge connecting the remaining two degree one vertices to complete a Hamilton circuit.*



Problem 23. *Solve the TSP for the weighted graph PQRST using the Brute Force Algorithm. Draw the complete tree diagram and use it to list all of the different Hamilton circuits. Next, compute the length of each circuit. Finally, tell which one is the shortest.*

Problem 24. *It was possible to use the Brute Force Algorithm in the previous problem since there were only 5 vertices. Suppose you want to use the Brute Force Algorithm to solve the TSP for a graph with 12 vertices, that you can compute the length of one Hamilton circuit*

	A	B	C	D	E	F	G	H	I	J	K	L
A		8	15	7	6	9	10	16	11	7	9	10
B			9	12	6	9	11	15	12	8	9	6
C				7	10	11	9	5	6	9	10	12
D					8	6	13	10	9	5	11	7
E						10	7	6	9	10	8	9
F							9	8	10	6	5	16
G								7	15	7	6	8
H									5	9	14	10
I										8	19	7
J											12	5
K												6

Table 2.1: table of distances between 12 houses

in 10 minutes, and that you are willing to work at it for 10 hours a day until you finish. How long will it take you to check all of the different circuits? (*You must choose the best units to use when you give your answer—minutes, hours, days, weeks,...—which ever is most appropriate.*)

Problem 25. *Use the Nearest Neighbor Greedy Algorithm to solve the TSP in the graph PQRST. You need to start by drawing 5 copies of the vertices of the above graph, one for each starting place. Circle the starting vertex of each one, and then insert the edges that you use for your circuit numbering them as you add them. Write under each one the length of the path you get. Finally, tell which path is the shortest. That is your solution to the TSP.*

Problem 26. *Use the Cheapest Link Greedy Algorithm to solve the TSP in the graph PQRST. Draw the 5 vertices, then add the edges one at a time, numbering them as you go, until you have a circuit. What is the length of this circuit?*

Problem 27. *The Greedy Algorithms* (*NN and CL*)*, like Fleury's Algorithm but unlike the Brute Force Algorithm, are very quick and efficient to apply. The problem with them is that, unlike Fleury's Algorithm, they don't always give us the shortest path! Find a* (*small*) *example of a weighted graph in which neither of the Greedy Algorithms produce a correct answer to the TSP.* (*This proves mathematically that greed does not always pay!*)

Problem 28. *Suppose that you need to make a delivery to each of 11 friends and then return home. What's more, your car is in very bad shape and you don't want to drive it any further than absolutely necessary. Call your house A and the houses of your friends B, C, D, E, F, G, H, I, J, K and L. Table 1 below indicates the distances between each pair of houses. You need to find a Hamilton circuit through the complete graph on 12 vertices, A, B, C, D, E, F, G, H, I, J, K and L, which has as small a total distance as possible. According to what you found in doing Problem 24, you are quite certain that you don't want to use the Brute Force Algorithm to find this circuit.*

1. *Pick a random Hamilton circuit and compute its total distance.*

2. *Apply the Nearest Neighbor Greedy Algorithm, starting from A (only), to find a Hamilton circuit. What is its total length?*

3. *Apply the Nearest Neighbor Greedy Algorithm, starting from D (only), to find a Hamilton circuit. What is its total length?*

4. *Apply the Cheapest Link Greedy Algorithm to find a Hamilton circuit. What is the length of this circuit?*

This last problem shows how the Greedy Algorithms can be efficient to use and can give a good solution, though they do not always give the best possible solution.

Problem 29. *Create from your own experience (real or fictitious) a problem that can be interpreted as finding a minimum weight Hamilton circuit in some weighted graph. Write the problem in detail and solve it using either the Nearest Neighbor Greedy Algorithm or the Cheapest Link Greedy Algorithm.*

Since 1971, when it first gained public attention, a vast effort has gone into finding a good solution to the Traveling Salesman Problem. To date, no solution has been found that is both quick and efficient to apply (like the Greedy Algorithms) and is guaranteed to always work (like the Brute Force Algorithm). In fact, researchers have uncovered considerable evidence that no such solution exists at all.

Chapter 3

Spanning Trees

Finding that you have no taste for either greed or brute force, you take a job as construction supervisor for the power company. The left diagram of Figure 3.1 shows 17 towns connected by roads. In the town at the bottom is a new solar power station, and it is your job to plan the construction of power lines that follow the roads and reach each town. Clearly, you don't need a power line constructed down every single road; you only need enough to ensure that there is a path from the power station to each town. The diagram on the right shows a way you might construct the power lines.

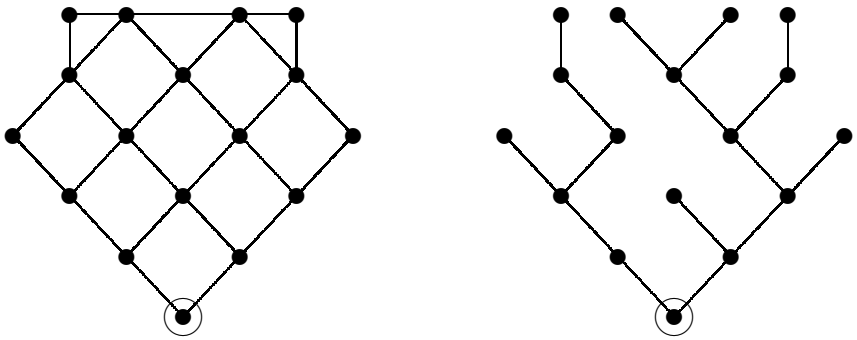


Figure 3.1: Spanning Tree

This looks like another opportunity to use your growing expertise in graph theory! In the language of graphs, the diagram on the left is a connected graph. The graph you seek, shown on the right, is a special subgraph. What is special about it? We call a connected graph which contains no circuits a **tree**. A **spanning tree** for a graph G is a tree whose vertices are the same as the vertices of G and whose edges are among the edges of G. In the example above, the graph on the right is a spanning tree for the graph on the left. We have already seen how trees are useful when we apply Brute Force Algorithms. There are

in fact several equivalent definitions of trees:

- a **tree** is a connected graph with no circuits;
- a **tree** is a graph in which there is exactly one path from each vertex to each other vertex that has no repeated edge;
- a **tree** is connected graph in which every edge is a cut edge.

The need for spanning trees can arise in a variety of different situations. One would be connecting towns to utility companies. As another, imagine a city with a limited number of working snow plows is hit with an unusually bad snow storm. They need to plow the streets so that people can get around. One approach would be to plow every street starting at one end and working their way, over a few days, to the other end. A better approach might be to look at the streets as a graph, the intersections as vertices. Then plow out a spanning tree which would allow people to get from any intersection to any other intersection in the entire city.

Finding a spanning tree is easy for a small graph like the one in Figure 3.1 that we can draw. In reality spanning trees are often needed for much larger graphs which are too big to draw. (Imagine connecting power lines to every town in the country.) In these cases we need an *algorithm* for constructing a spanning tree.

Spanning Tree Algorithm. *To construct a spanning tree for a connected graph G, make another copy of the vertices of G. Then*

1. *add edges from G between these vertices one at a time making sure to never close a circuit, and*
2. *stop when no additional edge from G can be added without closing a circuit.*

Proposition 4. *Given a connected graph G, the Spanning Tree Algorithm always produces a spanning tree for G.*

Proof: The result of the Spanning Tree Algorithm is a graph with the same vertices as the original graph but with no circuits. To see that it is a spanning tree, we must convince ourselves that it is connected. But as long as it is not connected, it is possible to add another edge between two components without creating a circuit. #

Problem 30. *Copy the vertices of the graph in Figure 3.1, and apply the Spanning Tree Algorithm to find a different spanning tree.*

Problem 31. *Draw ten trees. Under each one, write the number e of edges it has and then write the number v of vertices it has.*

Problem 32. *Looking at what you found in the previous problem, what do you think is the relationship between the number of vertices and the number of edges in a tree?*

In order to establish that your conjecture in Problem 32 is correct, recall that a **component** of a graph is a maximal connected subgraph. The graph in Problem 16 of Chapter 1, for example, has two components. Suppose we want to make a tree with 50 vertices. We start by drawing the 50 vertices. This gives us a graph with 50 components and no edges.

Now we add edges, one at a time, never closing a circuit. When we add an edge, what do we do to the number of components? So how many edges do we need to add to reduce the number of components to one? That will be the number of edges in our tree. We can state this conclusion in general.

Proposition 5. *A tree with v vertices has e = edges.*

In fact, the converse is easily seen to be true as well. For suppose a connected graph has 50 vertices and 49 edges. Must it be a tree? Using the Spanning Tree Algorithm, we could construct a spanning tree for the graph. According to Proposition 2, the spanning tree would also have 49 edges, so it would be exactly the original graph. This gives us a fourth equivalent description of trees:

- A **tree** is a connected graph where the number of edges is one less than the number of vertices.

Now look back at the problem stated at the beginning of this chapter. Using the Spanning Tree Algorithm, you could find many different spanning trees to use to build your power lines. Are some preferable to others? You would probably like to find the spanning tree that is most economical to construct. This might mean the one with the smallest total *length* of power lines, measured in miles. Alternatively, you might observe that it is more costly per mile to build roads over mountains than between corn fields. In this case you would want to estimate the cost of each road, and choose a spanning tree with the smallest total *cost*. In either case, the problem will be modeled mathematically with a **weighted graph**, as was the TSP. Weights, then, will be used to represent quantities like distances or costs, and our goal will be to solve the following problem.

Minimum Weight Spanning Tree Problem. *Given a weighted graph, find a spanning tree whose total weight is less than or equal to that of any other spanning tree.*

In the diagram below on the left, we have added weights to represent either distances (in miles) or road construction costs (in millions of dollars) between our 17 towns.

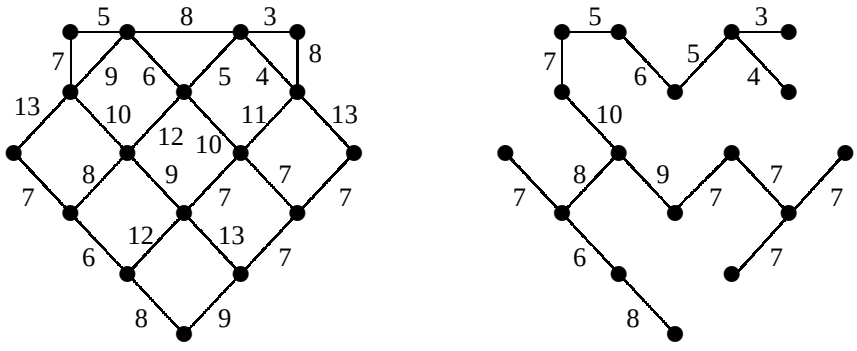


Figure 3.2: Weighted Graph

Using this weighted graph, we can calculate the total weight of the spanning tree in Figure 3.1 to be 129. Is this a minimum weight spanning tree? As it was constructed randomly using the Spanning Tree Algorithm, chances are that it is not. In fact it isn't, as we see from the alternative spanning tree on the right which has a total weight of only 106.

We can now ask if *this* spanning tree has minimal total weight among all possible spanning trees. This is not easy to see. One way to try to find out is to compare it with other spanning trees that we might construct. Like the other problems we have looked at, the difficulty of checking all possible spanning trees rises quickly as the size of the graph gets bigger. For large weighted graphs, a brute force algorithm is not a practical method. What we need is an efficient algorithm to find a minimum weight spanning tree.

One reason we like to study the minimum spanning tree problem is that there is in fact a wonderful algorithm to solve it. This algorithm, discovered by Joseph Kruskal in 1956, simply amounts to the Spanning Tree Algorithm enhanced by the Cheapest Link Greedy Algorithm. Instead of choosing an arbitrary edge that doesn't complete a circuit, we choose the cheapest (least weight) one. Since Kruskal's Algorithm is also a greedy algorithm, it is practical and efficient to use. But unlike our other greedy algorithms, it *always works*!

Kruskal's Greedy Algorithm. *To construct a minimum weight spanning tree for a connected weighted graph G, make another copy of the vertices of G. Then*

1. *add edges from G between these vertices one at a time, each time choosing from among the edges that do not close a circuit one of minimal weight, and*
2. *stop when no additional edge from G can be added without closing a circuit.*

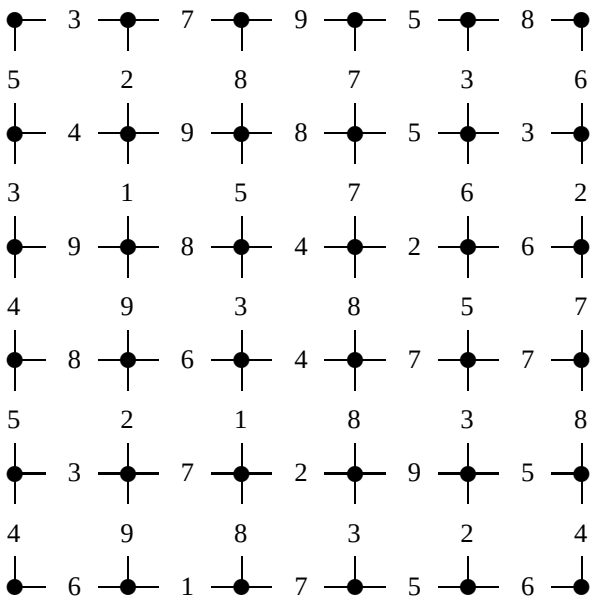
You can check that the 108-weight spanning tree above is obtained using Kruskal's Greedy Algorithm. Kruskal's important contribution was to show that his algorithm always produces a spanning tree of minimal possible total weight.

Kruskal's Theorem. *Kruskal's Greedy Algorithm always produces a minimum weight spanning tree T from a connected weighted graph G.*

Proof: (Outline) Suppose S is any spanning tree for G. We must convince ourselves that the total weight of S is at least as big as that of the spanning tree T obtained from Kruskal's Algorithm. Consider the first edge XY that we put in T but is not in S. Since S is already connected, adding XY to S must close a circuit (Why?). Since T is a tree, it contains no circuit so there is some edge UV in this circuit that is not in T. Now Weight(UV) must be at least as big as Weight(XY); otherwise Kruskal would have put UV into T instead of XY. Notice that if we replace UV by XY in S, then

- the total weight of S does not increase;
- S is still a spanning tree;
- we have increased the number of edges that S and T have in common.

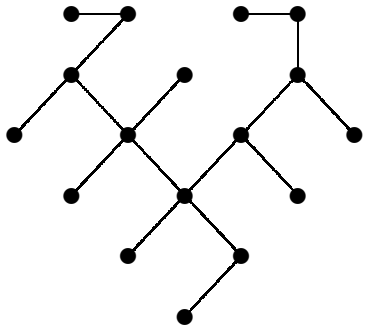
Repeating this process with the edges of T that are not in S, in the order they were added to T, we can slowly transform S into T without increasing its total weight. Thus the total weight of S could not have been less than that of T to begin with. #



Problem 33. *Draw the* 36 *vertices of the above weighted graph on a piece of paper. Now use the Spanning Tree Algorithm to construct an arbitrary spanning tree for the graph. What is the total weight of this arbitrary spanning tree?*

Problem 34. *Draw another copy of the vertices of this graph. This time apply Kruskal's Greedy Algorithm to find a minimum weight spanning tree. What is the total weight of this spanning tree?*

In the next two problems we will review the proof of Kruskal's Theorem to see why the spanning tree obtained from Kruskal's Algorithm always has the smallest total weight of any spanning tree. In the figure below is a quite arbitrary spanning tree S for the graph in Figure 3.1. We would like to see why its total weight must be no less that that of the spanning tree obtained from Kruskal's Algorithm.



Spanning Tree S

Problem 35. *Make a copy of the vertices of the weighted graph on the left side of Figure 3.2, and then apply Kruskal's Algorithm to obtain a minimum weight spanning tree T. As you do it, number the edges in the order you add them. Find the total weight of T.*

Problem 36. *What is the total weight of the spanning tree S? Make your own copy of S* in pencil. *Then apply the proof of Kruskal's Theorem to the spanning tree S to transform it into T by adding the edges of T to it, in the order you numbered them, each time removing an edge not in T that closes a circuit. As you do so, fill out a table like this:*

Weight of edge added to S from T	Weight of edge not in T removed from S	Amount the weight of S is reduced
...	...	...
...	...	...

When you have finished transforming S into T, add up the reductions you made in the weight of S. Now subtract this amount from the original weight of S to see if you get the weight of T. Write this out neatly *so that it is clear what you have done. (If you can reduce the weight of S to get the weight of T, then you know that T must have smaller weight than S!)*

Chapter 4

Scheduling Problems

Suppose that it takes 1000 man-hours to mow a certain golf course. That would mean that 1 person could mow it in a 1000 hours, or that 1000 people (with 1000 lawn mowers) could mow it in 1 hour. It would also mean that 50 people could mow it in 20 hours, or any number of other combinations.

Now suppose we found that building a certain house required 1000 man-hours. That would mean that 1 carpenter could build it in 1000 hours. Would it also mean that 1000 carpenters could build it house in 1 hour? Apparently not, as the one nailing on the roof, for example, couldn't start until many other steps had been completed. Some tasks could be done simultaneously, but others could not begin until certain previous ones were complete. We might expect that adding additional carpenters would reduce the time to complete the job up to a certain point, and beyond that point additional carpenters would not have anything to do.

Scheduling work for complex projects is a major task which is often not visible to the outside world. The goal of scheduling is to put together a work schedule which gets the project done in as little time as possible with the fewest workers necessary. The problem of mowing a field is easy, since we can have as many workers as we like, all mowing at the same time. But most projects, like building a house, require that we observe certain precedence relations: some tasks must be completed before others can start.

It turns out that there is some interesting, and quite recently developed, mathematics that can help construct the best schedule for a project. First, we need to be clear as to what is necessary to specify a particular project. The work will be done by workers who might be

- people (carpenters, lawn workers, cooks, ...),
- work crews (masons, framers, plumbers, roofers, ...),
- horses (or other beasts of burden),
- robots (either stationary or mobile),
- computers (connected together in a network) or
- computer chips (working simultaneously within one computer),

to name a few possible examples. The workers will be called **processors**, a name taken from the last example to emphasize that they need not be people. The specification of a project will include a list of the processors (e.g., Sally, George, Henry,...) available to do the work. In fact, the number of processors is sufficient; we don't really need their names. In our study, we will assume that *each task is done by a single processor without interruption* and that *each processor is capable of doing every task*; otherwise the problem becomes much more complex.

In order to specify a particular project, then, we need four pieces of information:

1. the **number of processors** available to us;
2. a list of **tasks** that have to be completed;
3. the **time** required for each task and
4. the necessary **precedence relations** between the tasks.

Precedence relations between tasks are best conveyed by a "graph" which has the tasks as vertices and arrows (instead of edges) to indicate that one task must be completed before another is started. We call this a **directed graph**, or **digraph** for short. The information required by the last three items above is best conveyed by a **project digraph** as shown in this simple example.

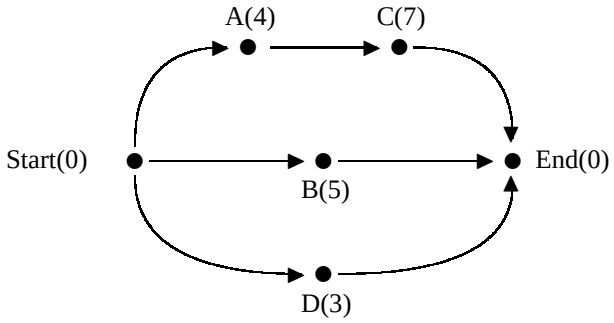
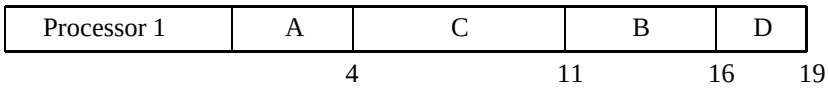


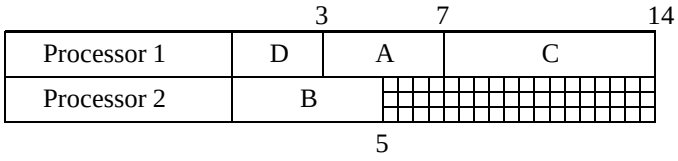
Figure 4.1: Project Digraph

This project has four tasks to be done, which we name A, B, C and D. Task A takes 4 hours to complete, B takes 5, C takes 7 and D takes 3. Task A must be completed before task C can be started; otherwise the tasks can be done in any order. For completeness, we always add a 0 time task $Start$, which precedes all other tasks, and a 0 time task End, which follows all other tasks.

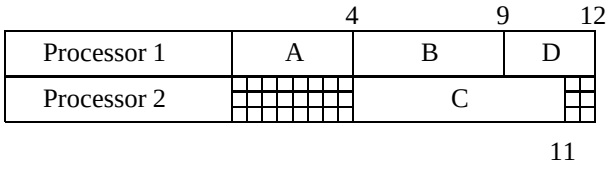
How might we schedule this project? Although it is very simple, it will illustrate the ideas we want to use. The first question is how many processors to use. There are a total of $4+7+5+3=19$ **processor hours** of work to be done. These can be done in several ways with a single processor, taking a total of 19 hours to complete, for example:



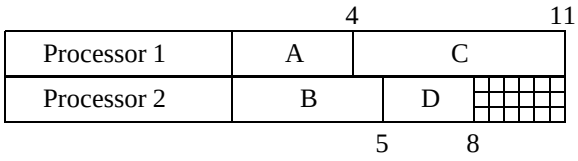
With two processors, we can reduce the **finishing time**, for example, to 14 hours with the schedule



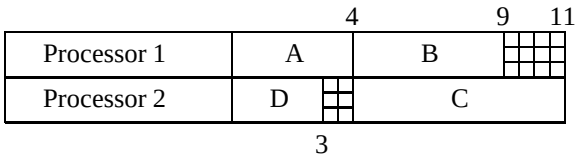
This is a very **inefficient schedule** with a great deal of wasteful **idle time**. We can do much better in 12 hours with the schedule



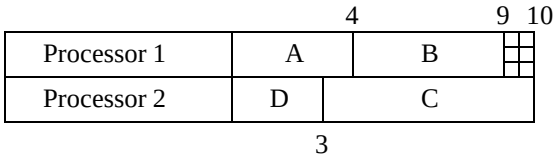
Can we do better? Just slightly, with either



or



to complete the project in 11 hours. If we cheat and construct an **illegal schedule** by starting *C* before *A* is complete, we can get the finishing time down to 10 hours!



In fact, the two 11 hour schedules are **optimal schedules** since the project cannot be legally completed in less than 11 hours. We see this by looking at the longest path

$$Start \Rightarrow A \Rightarrow C \Rightarrow End$$

from *Start* to *End*. This is called the **critical path**, and its length (11) is called the **critical time** of the project. Clearly, no project can be completed in less than its critical time. In this example, even additional processors would not increase the finishing time because we must wait for *A* to be completed before we start *C*. These observations lead us to the central problem we want to address in this chapter:

Scheduling Problem. *Given a project digraph and a list of processors, find an optimal schedule for the project.*

In order to produce a schedule, we must assign new tasks to the processors as they finish their previous tasks. At any particular time, we must examine the tasks to see which one to assign. Each task will be either

- **ready** if each preceding task is complete but it has not been started,
- **ineligible** if some preceding task is not complete,
- **in execution** if it is being done by some processor or
- **completed**.

We would like to put the schedule together by assigning to each free processor the *highest priority* task which is ready at that time.

This requires that we order the tasks in advance according to some **priority list**. A schedule will be completely determined once we have three ingredients:

1. the **project digraph**,
2. the **list of processors** and
3. the **priority list**.

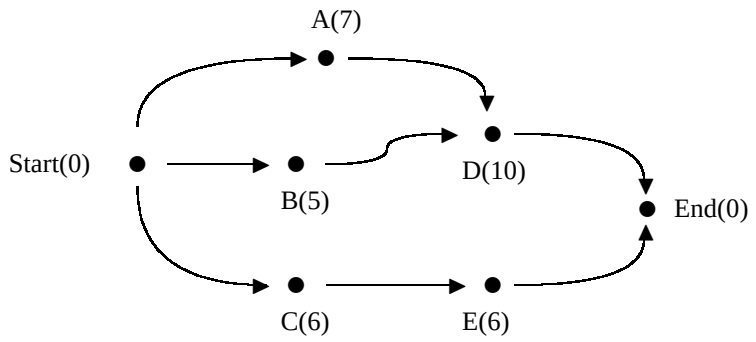


Figure 4.2: Five Task Project

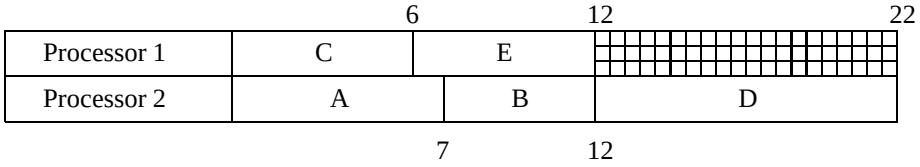
Once we are confident that we can produce a schedule from these three ingredients, we will address the real core of the problem: How do we come up with a good priority list?

It is time for some examples. Consider the project in Figure 4.2

The priority list

$$CEDAB$$

indicates that task C has the highest priority, E the next highest, D next, A next and B has the lowest priority. Using this priority list we get the schedule

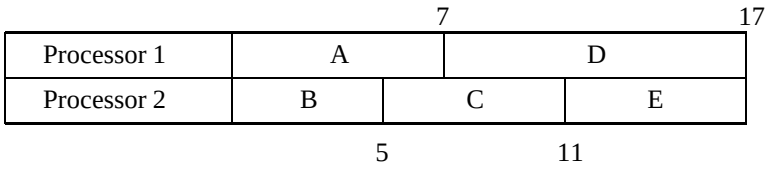


with a finishing time of 22 hours. The fact that Processor 1 is standing idle for 10 of those hours suggests that this is a very inefficient schedule. We also see this by looking at the critical path $Start \Rightarrow A \Rightarrow D \Rightarrow End$, which gives us a critical time of 17 hours—much less than 22.

Now try the priority list

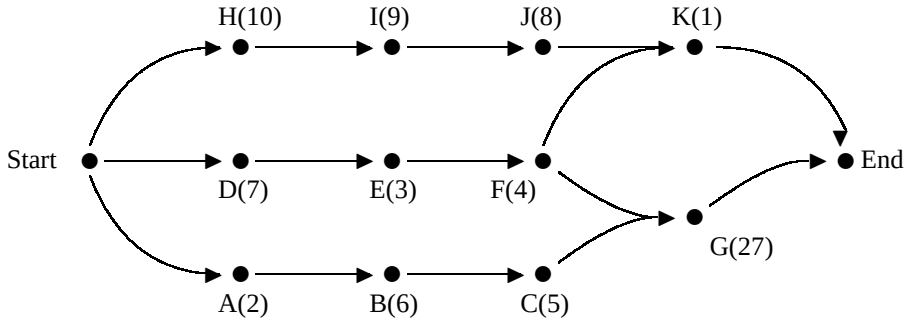
$$ADBCE.$$

We get instead the schedule



We can see that this is an optimal schedule in two ways. One is that there is absolutely no idle time. The other is that its finishing time is the same as the critical time of 17 hours.

You can now apply these ideas to the project specified by the following digraph.



Problem 37. *Schedule this project with 2 processors using the priority list*

GHIJDBCFEAK

Processor 1	
Processor 2	

Finishing time = ... Total idle time = ...

Problem 38. *Schedule this project with 2 processors using the priority list*

DABHCGFEIKJ

Processor 1	
Processor 2	

Finishing time = ... Total idle time = ...

Problem 39. *Schedule this project with 2 processors using the priority list*

DABECFHGIJK

Processor 1	
Processor 2	

Finishing time = ... Total idle time = ...

Problem 40. *Schedule this project with 3 processors using the worst priority list*

GHIJDBCFEAK

Processor 1	
Processor 2	
Processor 3	

Finishing time = ... Total idle time = ...

Problem 41. *What is the critical path for this project? What is the critical time? What would be an optimal number of processors and an optimal schedule for this project?* (*Explain your answer in complete sentences.*)

It is time now to look at the scheduling problem for a concrete project which is complex enough to make it difficult to pick out an optimal schedule by trial and error. It is reasonable to expect that the sale of gasoline combustion engines will be banned in the near future. Imagine that it is the year 2013 and that this ban has been imposed, effective January 1, 2014. An automobile company is producing a new fully recyclable "Eco-Car" with a hydrogen combustion – electric hybrid power system and an all aluminum chassis and frame. Demand is so great that they have a signed contracted for every car before it comes off the assembly line. They are intent on producing Eco-Cars as fast as possible so that they won't miss a sale. The construction of a car will require completing the following 15 tasks, each of whose times is given in the list below.

FC(6): attach Frame to Chassis.

AH(8): Assemble Hydrogen combustion engine.

IH(3): Install Hydrogen combustion engine and accompanying parts.

IC(5): Install pressurized hydrogen Cylinder.

IP(3): Install hydrogen Plumbing.

AM(1): Assemble electric Motor.

IM(2): Install electric Motor.

AG(4): Assemble electric Generator system.

TG(9): Test electric Generator system.

IG(6): Install electric Generator system.

IW(3): Install electric Wiring.

DW(10): Install doors, windows, and exterior panels.

IB(3): Install Braking system.

PE(11): Paint Exterior.

EE(12): install Everything Else.

Each Eco-Car will be assembled by two Mechanical Automatic Eco-Car Assembly Robots, or MAECARS (pronounced "make-cars") for short, that are each programmed to be able to do any one of these tasks. The scheduling of these tasks is constrained by the following diagraph giving the required precedence relations. For example, the can not install the wiring (IW) before both the hydrogen engine and the electric motor are installed [(IH) and (IM)].

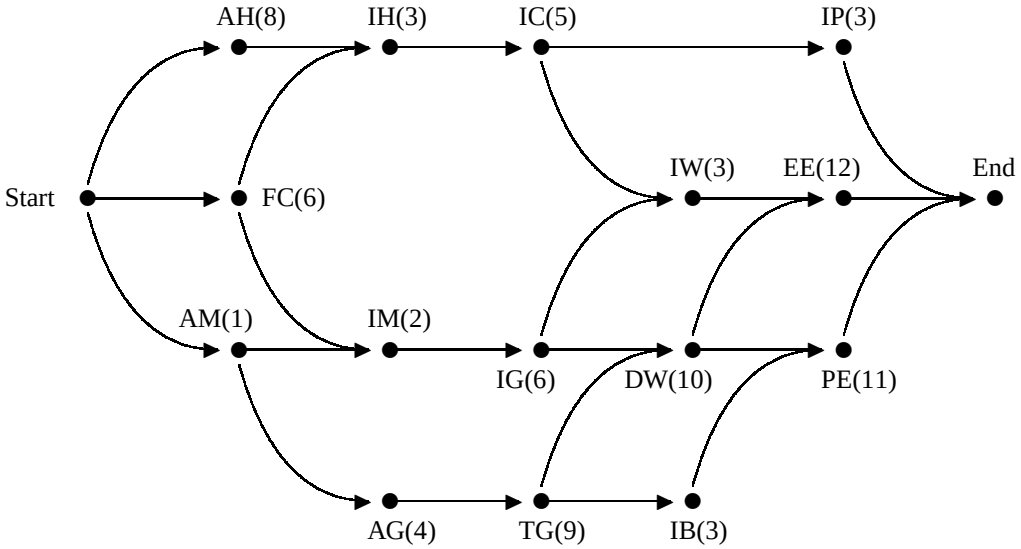
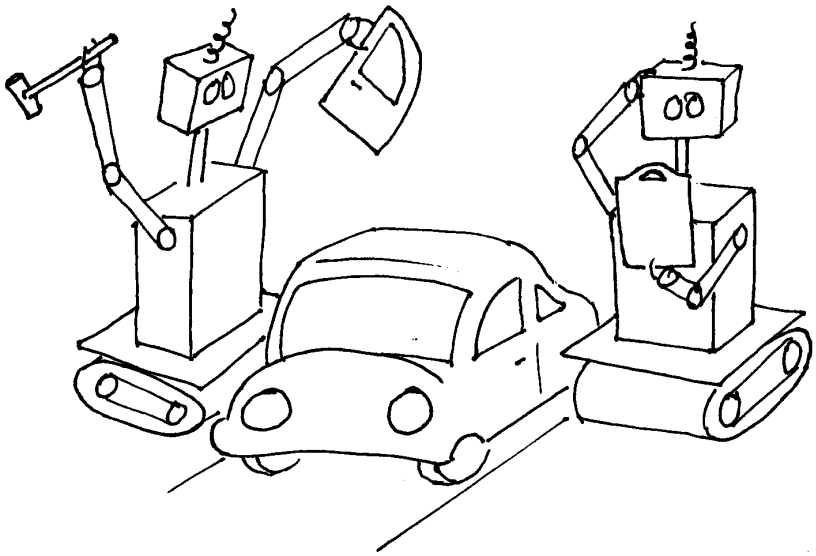


Figure 4.3: Assembly digraph for Eco-Car, executed by two MAECARS.



Our past experience tells us that the finishing time to assemble an Eco-Car is likely to depend on the priority list we use to make the schedule. A small savings in the time of construction of an expensive car can mean a substantial savings in price for the consumer, so we would like to find the best schedule possible.

We can try scheduling the project with two MAECARS by writing down a random priority list:

$$AH, IH, IC, IP, EC, IM, IW, IB, PE, AM, AF, TG, IG, DW, EE.$$

The resulting schedule includes 14 hours of idle time and has a finishing time of 50 hours. How good is this? Adding up the times of all the tasks, we see that the project requires a total of 86 MAECAR-hours work. If each MAECAR worked without any idle time, the project would be completed in 43 hours. So no schedule could complete the project in less than 43 hours. Given the large number of Eco-Cars that the company expects to sell, every hour that brings us closer to the 43 hour absolute minimum will represent a great savings.

These examples lead us to the central problem of scheduling:

Priority List Problem. *How can we find a good priority list to schedule a project?*

Remember that the project digraph, the list of processors and the priority list completely determine the schedule. The project digraph and the list of processors available are usually given to us, but we are free to devise our own priority list.

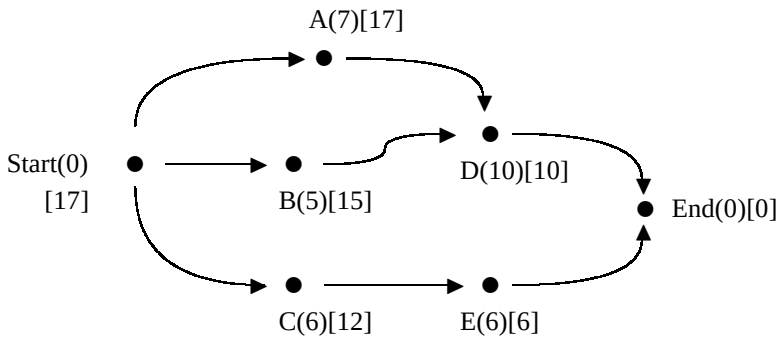
What is it that should give one task a higher priority than another? Our examples suggest that the tasks which take a *longer time* need to be completed as expediently as possible. This leads to a specific scheduling algorithm.

Decreasing Time Algorithm. *Construct a schedule with a priority list in which the tasks are ordered from the longest time to the shortest time.*

Look, for example, at our original problem given in Figure 4.1. The Decreasing Time Algorithm would give us the priority list C, B, A, D, which would lead to the first 11 hour schedule that we found. This, we saw, was an optimal schedule. Applying this algorithm to the Eco-Car project, we get a finishing time of 49 hours—a small improvement over 50 hours. But applying the Decreasing Time Algorithm to the project in Figure 4.2 gives us the priority list D, A, C, E, B which leads to the same very bad 22 hour schedule we got with C, E, D, A, B.

This last example shows us the problem with the Decreasing Time Algorithm. We need to complete the critical path $Start \Rightarrow A \Rightarrow D \Rightarrow End$ as directly as possible, and this will take 17 hours. But D isn't done until B is finished, so B should have a high priority. The fact that B takes a short time is irrelevant; what is relevant is that the shortest path from B to End is 15 hours. We call the path $B \Rightarrow D \Rightarrow End$ the **critical path from** B. Since it is longer than the critical path $C \Rightarrow E \Rightarrow End$ from C, we see that B should have higher priority than C.

Working from right to left, we can add in brackets [ ] the length of the critical path from each vertex. We call this the **critical time** of the task.



This method is called the **Backflow Algorithm.** It gives us three important pieces of information:

- the critical time of each task,
- a method to trace the critical path of the project itself and
- the critical time of the entire project (which is the critical time of the task *Start*).

In this example we see that the critical path of the project is $Start \Rightarrow A \Rightarrow D \Rightarrow End$ and the critical time of the project is 17 hours.

If we give highest priority to tasks with the greatest critical time, we get the priority list A, B, C, D, E. We can easily check that this list gives us the same 17 hour optimal schedule as we obtained from the list A, D, B, C, E.

Critical Path Algorithm. *Construct a schedule with the priority list obtained as follows.*

1. *Apply the Backflow Algorithm to find the critical time of each task.*
2. *List the tasks in decreasing order of critical time.*

Try applying the Critical Path Algorithm to the Eco-Car project. You should get a finishing time of 44 hours, a big improvement over 49 hours and very close to the theoretical limit of 43 hours.

The Critical Path Algorithm is the most successful general algorithm for scheduling. It is efficient and practical, and is commonly used today. It remains only for us to find out if it always gives an optimal schedule.

Consider the project specified by the digraph of Figure 4.4.

Problem 42. *Schedule this project with 2 processors using the Decreasing Time Algorithm.*

Priority List:

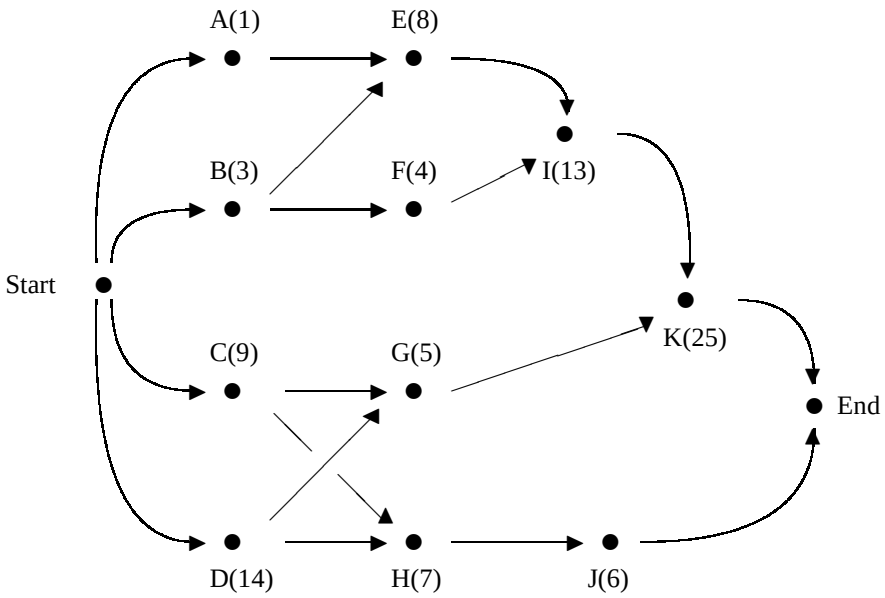


Figure 4.4: Eleven Task Project

Processor 1	
Processor 2	

Finishing time = ... Total idle time = ...

Problem 43. *Schedule the same project with 2 processors using the Critical Path Algorithm.*

1. *Use the Backflow Algorithm to find the critical time for each task.*

1. *What is the critical path of the project?*
2. *What is the critical time of the project?*
3. *What is the critical time priority list?*
   Priority List:

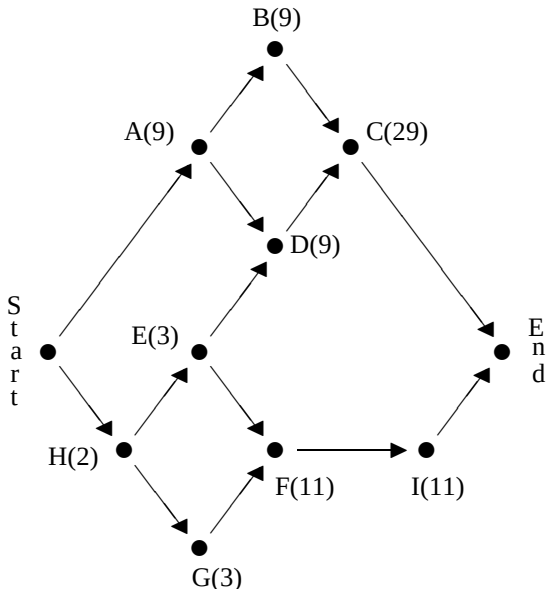
4. *Construct the critical path schedule:*

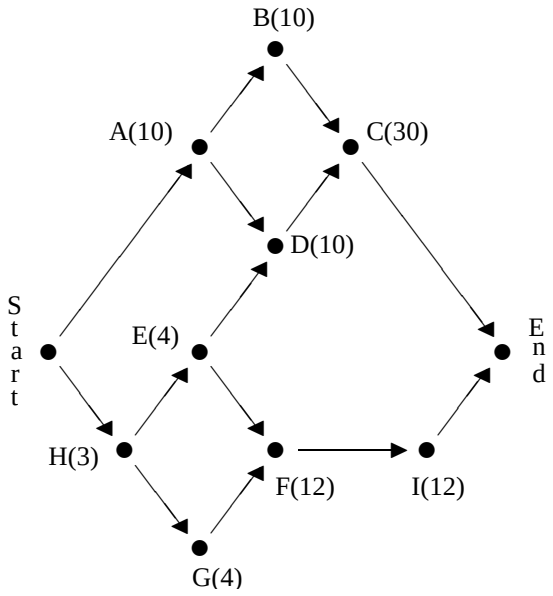
Processor 1	
Processor 2	

Finishing time = ... Total idle time = ...

The Hares and the Tortoises

A couple of hares, H1 and H2, decide to build a hare house together. Across the meadow a couple of tortoises, T1 and T2, also decide to build a house together. Although the two houses will have somewhat different styles, they both require the same 9 basic tasks described in the project digraphs shown below. As you will notice from the digraphs (though you probably already know), tortoises are slightly slower than hares. In fact, it takes a tortoise exactly *one hour more* to complete any particular task than it takes a hare.





Problem 44. ***Hare Schedule***

1. *Apply the Backflow Algorithm to find the critical time of each task in the Hare Digraph. Mark a critical path of the project. What is the critical time for the entire project?*

2. *Make a critical time priority list:*

3. *Construct a schedule for the hares. What is their finishing time?*

Hare 1	
Hare 2	

Problem 45. ***Tortoise Schedule***

1. *Apply the Backflow Algorithm to find the critical time of each task in the Tortoise Digraph. Mark a critical path of the project. What is the critical time for the entire project?*
2. *Make a critical time priority list:*
3. *Construct a schedule for the tortoises. What is the finishing time?*

Tortoise 1	
Tortoise 2	

Problem 46. *If you did everything correctly, you will have discovered something strange: the tortoises finished their house faster than the hares!*

How can this be? The problem is that the Critical Path Algorithm didn't produce a very good schedule for the hares. Surely, if each hare can complete each task faster than each tortoise, then the hares together should be able to complete the entire project faster than the tortoises! Show that this is true by using the tortoises' schedule and the hares' task times to construct an alternate schedule for the hares that will have a shorter finishing time than that of the tortoises. [*Remember that you aren't using any algorithm to produce this schedule. Just use your own good judgement.*]

Hare 1	
Hare 2	

What is the finishing time for this schedule? Is it optimal? (*Explain your answer in a complete sentence.*)

These three problems demonstrate two interesting facts:

1. When you apply the Critical Path Algorithm, using faster processors does not necessarily guarantee a shorter finishing time.
2. **The Critical Path Algorithm, like the Nearest Neighbor Algorithm and the Cheapest Link Algorithm, does not always give an optimal solution.**

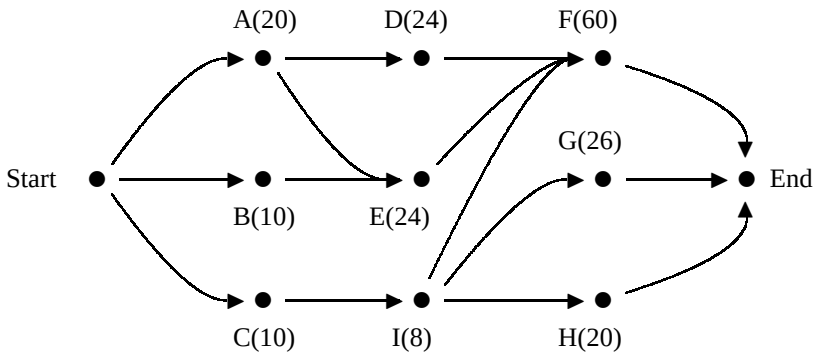
Nevertheless, the Critical Path Algorithm remains the best general algorithm for job scheduling. Often it gives us a good schedule which then, by some ad-hoc tinkering, can be made into a better schedule.

Too Many Cooks

Our experience with scheduling suggests that adding more processors will in general decrease our finishing time up to a point. Beyond that point we cannot keep more processors busy, so more processors only increase idle time without changing the finishing time.

Do you think that it is ever possible for more processors to result in a *longer* finishing time?

The Queen's Head Chef doesn't think so. He plans a big soup for the Queen's banquet which requires that nine different tasks be completed according to the task digraph below. Times are given in minutes.



Problem 47. *Compute the critical time of each task to obtain a priority list:*

What is the critical time of the project?

Problem 48. *The Head Chef decides to hire two cooks, C1 and C2, to make the soup, and asks you to use the Critical Path Algorithm to produce a schedule.*

C1	
C2	

What is the finishing time?

Problem 49. *Seeing that the finishing time exceeds the critical time, the Head Chef rips your schedule to pieces. "I'll hire three cooks!" he shrieks. Make a new schedule for three cooks, C1, C2 and C3.*

C1	
C2	
C3	

What is the finishing time now?

Problem 50. *Now ignore the algorithms and the priority lists.*

1. *Find a schedule for 3 cooks with a finishing time that is SHORTER than the finishing time of the schedule produced by the Critical Path Algorithm.*

C1	
C2	
C3	

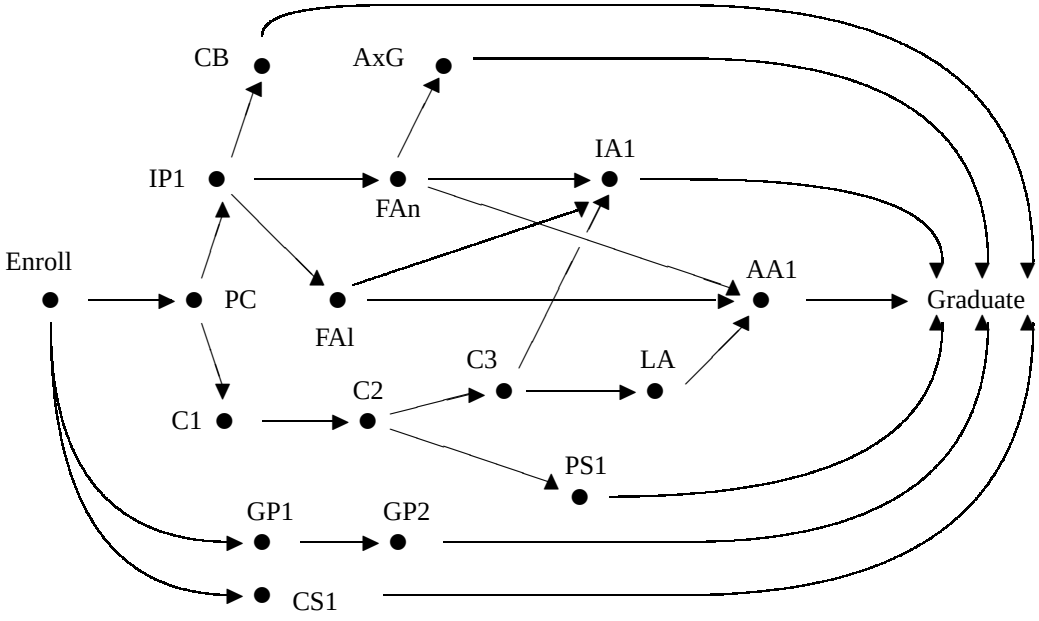
2. *Explain why this schedule must be optimal.*

Course Schedules

In the past we have thought of processors as workers or machines which do particular tasks. This problem shows how our methods can model a very different kind of processor.

Colleges often enroll students who already have a bachelors degree and come to earn a second bachelors in a different discipline. In order to do this, a student need only complete the courses required for the new major. Such students normally work at least part time and take 1, 2 or 3 courses at a time.

Imagine, for example, that you have a friend who completes a BA in English, Secondary Education and decides to return for a BA in Mathematics, Secondary Education. The diagram below lists the courses she must complete, with arrows to indicate prerequisites. For example, PC is a prerequisite for C1.



[PC - Precalculus; IP1 - Introduction to Proofs 1; FAn - Foundations of Analysis; FAl - Foundations of Algebra; C1,C2,C3 - Calculus 1,2,3; CS1 - Computer Science 1; GP1,GP2 - General Physics 1,2; CB - Combinatorics; AxG - Axiomatic Geometry; PS - Probability and Statistics 1; IA1 - Intermediate Analysis 1; LA - Linear Algebra; AA1 - Abstract Algebra 1]

What you need to do is to construct a schedule telling her, semester by semester, which courses to take. She can only take a course after she has completed its prerequisite courses.

Problem 51. *At first she would like to consider enrolling full time in order to finish quickly, taking up to 5 different courses each semester. Think of a processor as one of the time slots in her schedule, and measure time in semesters. Use the Critical Path Algorithm to construct a schedule.*

T1								
T2								
T3								
T4								
T5								
	1	*2*	*3*	*4*	*5*	*6*	*7*	*8*

semester number

Priority List:

Problem 52. *Looking at this schedule, you suggest she see how much longer it would take to finish if she got a job and took at most 2 courses at a time. Use the Critical Path Algorithm to schedule the same curriculum assuming that she will take at most two different courses each semester.*

T1								
T2								
	1	*2*	*3*	*4*	*5*	*6*	*7*	*8*

semester number

Priority List:

Problem 53. *Compare the finishing times of these two schedules. Then explain exactly which alternative you would advise your friend to take, giving clear reasons as to why.*

How bad can it be?

Given a particular project, we have seen examples of good schedules (usually coming from the Critical Path Algorithm) and bad schedules (coming from a bad priority list). Judging a schedule by its finishing time, how much worse can a bad schedule be than the optimal schedule? Can it take twice as long? three times as long? ten times as long?

To make the question more specific, let T_{op} be the finishing time of an optimal schedule for a particular project. Thus T_{op} is the shortest time in which the project can be completed. Now consider any crazy priority list that someone might make up, and let T be the finishing time of the resulting schedule. How big can the ratio $\frac{T}{T_{op}}$ be? (For example, if $\frac{T}{T_{op}} = 3$, then the finishing time of the crazy schedule is 3 times as long as that of the optimal schedule.)

This question was answered by Ronald Graham of AT&T Bell Laboratories in 1966. He discovered that the answer depends on the number M of processors used, but that the finishing time T can never be more than *twice* as long as the optimal time T_{op}.

Graham's Theorem. (1966) *If a priority list gives a schedule with finishing time T using M processors, then*

$$\frac{T}{T_{op}} \leq 2 - \frac{1}{M},$$

that is, the finishing time T is ***at most*** $2 - \frac{1}{M}$ *times the finishing time T_{op} of the optimal schedule.*

For example, if $M = 2$ processors are used, then T is never more than $2 - \frac{1}{2}$ or $1\frac{1}{2}$ times as long as the optimal time. We will not prove Graham's Theorem, but we will show that schedules can be made that are every bit as bad as he says they might be.

Problem 54. $(M = 2)$: *Can T ever be $2 - \frac{1}{2} = 1\frac{1}{2}$ times as big as T_{op} with 2 processors? Consider a project with tasks $A(1)$, $B(1)$, $C(1)$, $D(1)$, $X(4)$ and* no *precedence relations*

that you would like to schedule with 2 *processors. Find the critical time of the project. Experiment with different priority lists to find an optimal schedule that finishes this project in the critical time.*

Critical time:

Optimal priority list:

Optimal schedule:

P1	
P2	

Finishing time T_{op}*:*

Now find a bad *priority list that produces a schedule with a finishing time that is* $2-\frac{1}{2}=1\frac{1}{2}$ *times as long.*

Bad priority list:

Worst possible schedule:

P1	
P2	

Problem 55. $(M=3)$: *Can* T *ever be* $2-\frac{1}{3}=1\frac{2}{3}$ *times as big as* T_{op} *with* 3 *processors? Consider a project with tasks* $A(1), B(1), C(1), D(1), E(1), F(1), Y(3)$ *and no precedence relations that you would like to schedule with* 3 *processors.*

1. *What is the critical time of the project?*
2. *Find an optimal priority list that finishes in the critical time.*
3. *Use this priority list to construct an optimal schedule.*
4. *Now find a* bad *priority list that produces a schedule with a finishing time that is* $2-\frac{1}{4}=1\frac{3}{4}$ *times as long.*
5. *Use this priority list to construct the worst possible schedule.*

Optimal schedule:

P1	
P2	
P3	

Worst possible schedule:

P1	
P2	
P3	

Problem 56. $(M = 5)$: *Can T ever be $2 - \frac{1}{5} = 1\frac{4}{5}$ times as big as T_{op} with 5 processors? This time you design the project! Give a list of the tasks with their times:*

1. *Give a list of the tasks with their times.*
2. *What are the precedence relations?*
3. *What is the critical time of the project?*
4. *Find an optimal priority list that finishes in the critical time.*
5. *Use this priority list to construct an optimal schedule.*
6. *Now find a* bad *priority list that produces a schedule with a finishing time that is $2 - \frac{1}{5} = 1\frac{4}{5}$ times as long.*
7. *Use this priority list to construct the worst possible schedule.*

Optimal schedule:

P1	
P2	
P3	
P4	
P5	

Worst possible schedule:

P1	
P2	
P3	
P4	
P5	

Problem 57. *Can T ever be $2 - \frac{1}{M} = \frac{2M-1}{M}$ times as big as T_{op} with M processors? (Here M is an arbitrary positive integer. For example, if $M = 100$, this would say that T could be*

$2-\frac{1}{100}=1.99$ *times as big as* T_{op} *with* M *processors. By choosing large values of* M*, this would allow us to make the ratio* $\frac{T}{T_{op}}$ *as close to 2 as we like.*)

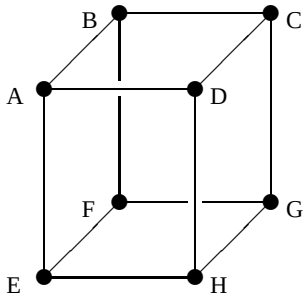
Again, you design the project!

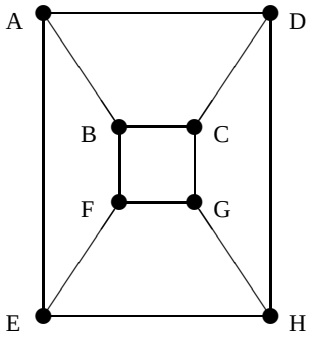
1. *Give a list of the tasks with their times.*
2. *What are the precedence relations?*
3. *What is the critical time of the project?*
4. *Find an optimal priority list that finishes this project in the critical time.*
5. *Use this priority list to construct an optimal schedule.*
6. *Now find a* bad *priority list that produces a schedule with a finishing time that is* $2-\frac{1}{M}$ *times as long.*
7. *Use this priority list to construct the worst possible schedule.*

Chapter 5

Planar Graphs

In this chapter we will study a variety of problems about graphs that can be drawn without their edges crossing. Try, for example, to draw the complete graph K_6 on 6 vertices without crossing edges. It's not so easy, right? The graph drawn on the left below, with edges crossing, is redrawn on the right without edges crossing.

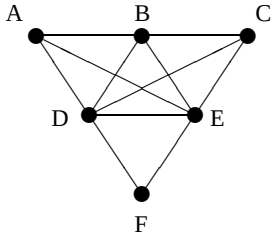


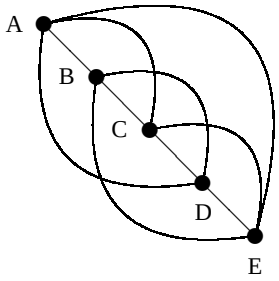


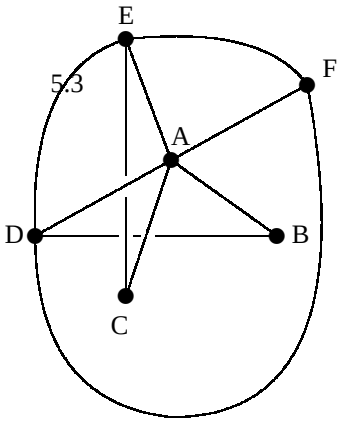
Redraw three of the four graphs below on a separate piece of paper without edges crossing. (Sometimes it helps to think of the edges as rubber bands that can be stretched into another position.)

Problem 58.

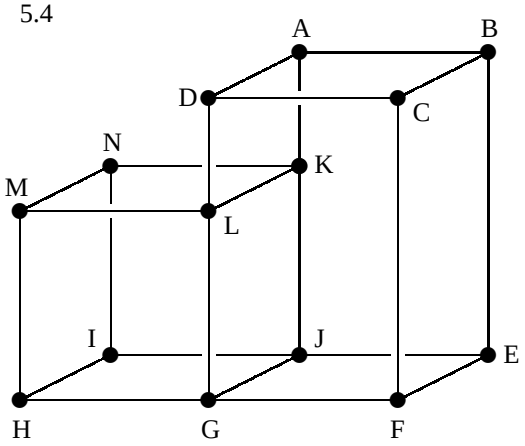
Problem 59.


A
B
C
D
E
F


A
B
C
D
E



5.3



5.4

Problem 60.

Problem 61.

What is the matter with the graph you couldn't draw? Is there a way to demonstrate that it can't be drawn without edges crossing? The first thing we need to observe is that this problem has to do with the particular **space** that we are making it in, in this case the surface of a piece of paper. If we tried to construct this graph, or K_6, in three dimensional space using wires instead of lines, for example, it would be easy to prevent edges from crossing.

In Chapter 1 we gave a definition of the word "path" that referred to a very special way of traveling through a graph. In this chapter we will use "**path**" in the common general sense, referring for example to the track of an ant across a piece of paper or a mosquito through the room. There is a special property of spaces like the surface of a paper that determines which graphs can be constructed in them.

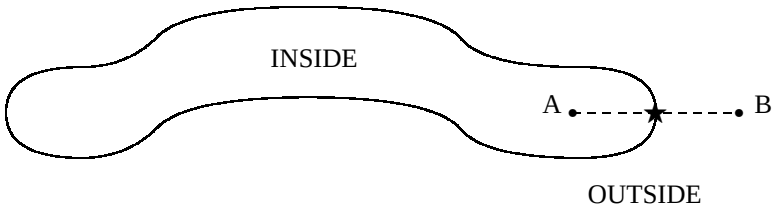


Figure 5.1: a simple closed curve

This property depends on the notion of a **simple closed curve**, that is, a path that starts and ends in the same place without crossing itself. A space is said to have the **separation property** (SP) if every simple closed curve in the space divides the space into two non-empty regions with no points in common, the **inside** and the **outside**, in such a way that every path starting on the inside and ending on the outside must cross the curve. Not all spaces have the SP. Can you determine which of the following spaces have this property?

- one side of a sheet of paper;

- the surface of a ball (a sphere);
- the space inside of an empty room;
- an infinite plane (like the Cartesian coordinate plane);
- the surface of a donut (a torus).

We will see how the SP will help us determine which graphs can be drawn on a piece of paper. These are called **planar graphs**. What we conclude from the SP will apply equally well to any other surface that has this property. We first need to establish two preliminary facts.

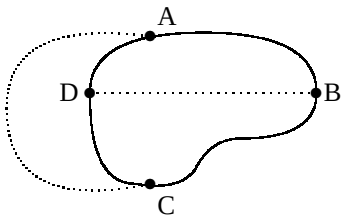
Proposition 6. *If a space has the SP, then every path connecting two points on a simple closed curve, and not otherwise intersecting the simple closed curve, lies entirely inside or entirely outside the curve.*

Proposition 7. *Assume a space has the SP. If A, B, C and D are points lying on a simple closed curve in that order and*

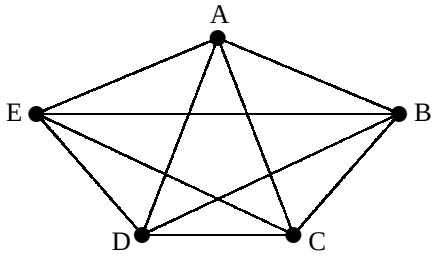
- *one path connects A and C,*
- *another connects B and D, and*
- *the two paths do not intersect each other or the simple closed curve between their endpoints,*

then one path must lie entirely inside the curve while the other is entirely outside the curve.

(Edges AC and BD can't be both drawn inside or both drawn outside without crossing.)



We can use Propositions 6 and 7 to demonstrate, for example, that the complete graph K_5 on 5 vertices cannot be drawn in the plane without edges crossing.



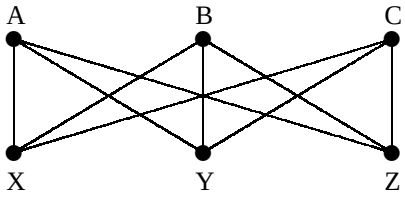
Take A, B, C, D and E to be the vertices, and suppose that someone tried to draw the graph without edges crossing. Then the path ABCDEA would be a simple closed curve. Edge AC would lie entirely inside or entirely outside this simple closed curve. First suppose that it is inside.

1. By the Proposition 6, edge BD must lie entirely outside.
2. By the Proposition 6, again, edge CE must lie entirely inside.
3. By the Proposition 6, again, edge DA must lie entirely outside.
4. By the Proposition 6, again, edge EB must lie entirely inside.
5. By the Proposition 6, again, edge AC must lie entirely outside.

This is a contradiction! We get a similar contradiction if we assume that edge AC lies entirely outside. Consequently it is not possible to draw K_5 in the plane without edges crossing.

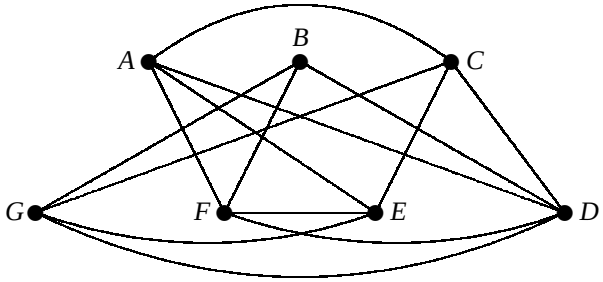
Problem 62. *Use Propositions 6 and 7 to demonstrate that one of the six graphs at the beginning of this chapter cannot be drawn without edges crossing.*

Problem 63. *Houses A, B and C each need to be connected by appropriate lines to the Power Company (X), the Gas Company (Y) and the Electric Company (Z). Use Propositions 6 and 7 to demonstrate that this cannot be done without any of these lines crossing.*



$G_{(3,3)}$

Problem 64. *Show that the graph below is planar by using what you've learned. You need to find a large circuit in the graph and draw it as a simple closed curve. Then use Propositions 6 and 7 to find out where the remaining edges must be drawn.*

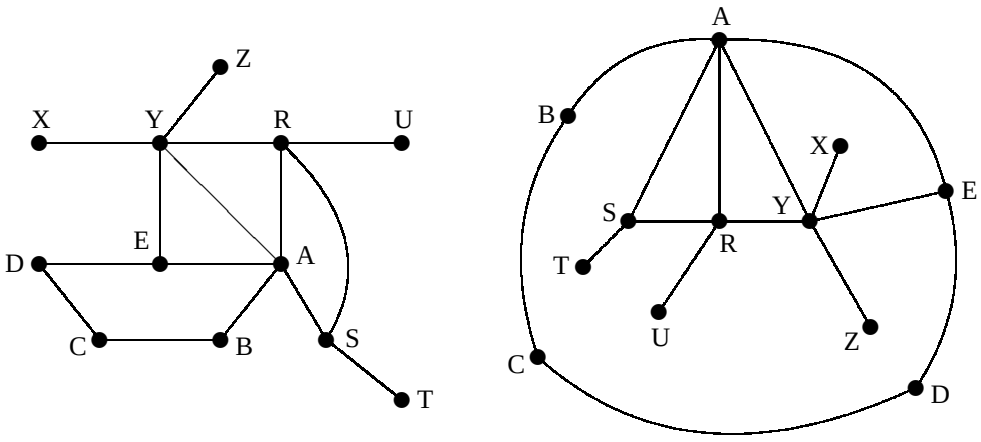


Problem 65. *The torus does not have the Separation Property, and consequently the arguments that K_5 and $G_{3,3}$ can't be drawn without edges crossing do not apply to the torus. Make two toruses from strips of paper, and draw these two graphs on them without edges crossing.*

Problem 66. *Draw the complete graph K_7 on seven vertices on the torus without edges crossing.*

Planar graphs are important because they are the graphs that we can most easily illustrate and understand. A printed circuit board is a good example of a complex graph that must, nevertheless, be planar. Every planar graph divides the plane it is drawn in into disjoint connected regions called the **faces** of the graph. Notice that there is always one large outside face. Below are two different illustrations of a particular connected planar graph G_1 with 12 vertices, 5 faces and 15 edges.

In the two drawings below, we see that different faces play the role of the outside face. In general we denote by v the number of **vertices** in a planar graph; by f the number of **faces** and by e the number of **edges**.



Problem 67. *On a separate sheet of paper, draw seven different connected planar graphs. Label them G_1, G_2, G_3, G_4, G_5, G_6 and G_7. Then tabulate the numbers f, v and e for each graph. When you are finished examine your answers and look for a relationship between these values that would allow you to predict one of them if someone tells you the other two.*

	G_1	G_2	G_3	G_4	G_5	G_6	G_7
f							
v							
e							

Problem 68. *I have a connected planar graph with 247 vertices and 192 faces. How many edges does it have? (Explain your answer!)*

Is there a way that you can know for sure if your guess is correct? We will now see that there is.

Problem 69. *On a separate sheet of paper draw a connected planar graph G with exactly 9 edges. You may include in it a loop (edge connecting a vertex to itself) and a multiple edge (more than one edge between the same two vertices). Be sure that it has several different faces. Now, on another sheet,"grow" the same graph as a sequence of 10 partial connected planar graphs labled G_1, G_2, G_3, G_4, G_5, G_6, G_7, G_8, G_9, $G_{10} = G$, like this.*

1. *Start with a single vertex as G_1.*
2. *Construct each graph from the previous one doing one of the following:*
   (a) *Add a new vertex and a new edge connecting it to an old vertex.*
   (b) *Add a new edge connecting two existing vertices.*

Examine each of your graphs and then fill in the table below.

	G_1	G_2	G_3	G_4	G_5	G_6	G_7	G_8	G_9	G_{10}
(a) or (b)?										
f										
v										
e										
$f+v-e$										

Problem 70. *Do each of the following.*

1. *Tell what the value of $f+v-e$ is for a single vertex (G_1).*
2. *Tell how the construction $2(a)$ affects the value of each of f, v, e and $f+v-e$.*
3. *Tell how the construction $2(b)$ affects the value of each of f, v, e and $f+v-e$.*

Problem 71. *You have now proven a famous theorem of Euler. (Fill in the blank.)*

Euler's Theorem *If a connected planar graph has f faces, v vertices and e edges, then*

$$f+v-e= \quad .$$

Index